

**LAPORAN AKHIR PRAKTIKUM
ALGORITMA & STRUKTUR DATA**



Oleh:

Bi'ahlil Haq Aulia Akbar Awaludin

NIM. 2010817110011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
2026**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM ALGORITMA & STRUKTUR
DATA

Laporan Praktikum Algoritma & Struktur Data

Modul 1 : Struct dan Pointer

Modul 2 : Stack dan Queue

Modul 3 : Linked List

Modul 4 : Sorting

Modul 5 : Searching

Modul 6 : Tree and Graph

ini disusun sebagai syarat lulus mata kuliah Praktikum Praktikum Algoritma & Struktur Data. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Bi'ahlil Haq Aulia Akbar Awaludin

NIM : 2010817110011

Menyetujui,

Asisten Praktikum

Mengetahui,

Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani

NIM. 2310817310009

Muti'a Maulida, S.Kom., M.TI.

NIP. 198810272019032013

DAFTAR ISI

LEMBAR PENGESAHAN.....	ii
DAFTAR ISI.....	iii
DAFTAR GAMBAR.....	vi
DAFTAR TABEL.....	vii
MODUL 1: STRUCT DAN POINTER.....	1
SOAL 1.....	1
A. Source Code.....	1
B. Output Program.....	2
C. Pembahasan.....	2
SOAL 2.....	3
A. Source Code.....	3
B. Output Program.....	4
C. Pembahasan.....	4
MODUL 2: STACK DAN QUEUE.....	6
SOAL 1.....	6
A. Source Code.....	6
B. Pembahasan.....	7
SOAL 2.....	9
A. Source Code.....	10
B. Output Program.....	12
C. Pembahasan.....	12
SOAL 3.....	14
A. Source Code.....	14
B. Pembahasan.....	15
SOAL 4.....	18
A. Source Code.....	19
B. Output Program.....	20
C. Pembahasan.....	20
MODUL 3: LINKED LIST.....	22

SOAL 1.....	22
A. Source Code.....	22
B. Pembahasan.....	26
SOAL 2.....	28
A. Source Code.....	29
B. Output Program.....	30
C. Pembahasan.....	31
SOAL 3.....	32
A. Source Code.....	32
B. Pembahasan.....	36
SOAL 4.....	38
A. Source Code.....	39
B. Output Program.....	41
C. Pembahasan.....	41
MODUL 4: SORTING.....	43
SOAL 1.....	43
A. Source Code.....	44
B. Pembahasan.....	51
MODUL 5: SEARCHING.....	63
SOAL 1.....	63
A. Source Code.....	65
B. Pembahasan.....	66
MODUL 6: TREE AND GRAPH.....	71
SOAL 1.....	71
A. Source Code.....	73
B. Output Program.....	74
C. Pembahasan.....	74
SOAL 2.....	76
A. Source Code.....	77
B. Output Program.....	79
C. Pembahasan.....	79
SOAL 3.....	80

A. Source Code.....	82
B. Output Program.....	83
C. Pembahasan.....	83
SOAL 4.....	86
A. Source Code.....	88
B. Output Program.....	89
C. Pembahasan.....	89
SOAL 5.....	91
A. Source Code.....	92
B. Output Program.....	95
C. Pembahasan.....	95
TAUTAN GIT.....	97

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 Modul 1.....	2
Gambar 2. Screenshot Hasil Jawaban Soal 2 Modul 1.....	4
Gambar 3. Screenshot Hasil Jawaban Soal 2 Modul 2.....	12
Gambar 4. Screenshot Hasil Jawaban Soal 4 Modul 2.....	20
Gambar 5. Screenshot Hasil Jawaban Soal 2 Modul 3.....	30
Gambar 6. Screenshot Hasil Jawaban Soal 4 Modul 3.....	41
Gambar 7. Screenshot Hasil Jawaban Soal 1 Modul 6.....	74
Gambar 8. Screenshot Hasil Jawaban Soal 2 Modul 6.....	79
Gambar 9. Screenshot Hasil Jawaban Soal 3 Modul 6.....	83
Gambar 10. Screenshot Hasil Jawaban Soal 4 Modul 6.....	89
Gambar 11. Screenshot Hasil Jawaban Soal 5 Modul 6.....	95

DAFTAR TABEL

Table 1. Contoh input dan output soal 1 Modul 1.....	1
Table 2. Source Code line.cpp.....	1
Table 3. Source Code prob1.cpp.....	2
Table 4. Contoh input dan output soal 2 Modul 1.....	3
Table 5. Source Code circle.cpp.....	3
Table 6. Source Code prob2.cpp.....	4
Table 7. Source Code stack.cpp.....	7
Table 8. Contoh input dan output soal 2 Modul 2.....	10
Table 9. Source Code prob1.cpp.....	11
Table 10. Source Code queue.cpp.....	15
Table 11. Contoh input dan output soal 4 Modul 2.....	19
Table 12. Source Code prob2.cpp.....	20
Table 13. Source Code single_linked_list.cpp.....	26
Table 14. Contoh input dan output soal 2 Modul 3.....	29
Table 15. Source Code prob_a.cpp.....	30
Table 16. Source Code double_linked_list.cpp.....	36
Table 17. Contoh input dan output soal 4 Modul 3.....	39
Table 18. Source Code prob_b.cpp.....	41
Table 19. Source Code sorting_algorithms.cpp.....	51
Table 20. Table waktu eksekusi Modul 4.....	59
Table 21. Table algoritma tercepat Modul 4.....	60
Table 22. Table algoritma sensitif bentuk data Modul 4.....	60
Table 23. Table algoritma sensitif bentuk data waktu Modul 4.....	61
Table 24. Table hubungan teori dan hasil Modul 4.....	61
Table 25. Source Code searching_algorithms.cpp.....	66
Table 26. Contoh linear search rekursif.....	67
Table 27. Contoh Binary search rekursif.....	69
Table 28. Table hasil experiment Modul 5.....	69
Table 29. Contoh input dan output soal 1 Modul 6.....	73
Table 30. Source Code soal1.cpp Modul 6.....	74
Table 31. Contoh input dan output soal 2 Modul 6.....	77
Table 32. Source Code soal2.cpp Modul 6.....	78
Table 33. Contoh input dan output soal 3 Modul 6.....	82
Table 34. Source Code soal3.cpp Modul 6.....	83
Table 35. Contoh input dan output soal 4 Modul 6.....	87
Table 36. Source Code soal4.cpp Modul 6.....	89
Table 37. Contoh input dan output soal 5 Modul 6.....	92
Table 38. Source Code soal5.cpp Modul 6.....	95

MODUL 1: STRUCT DAN POINTER

SOAL 1

Diberikan sebuah garis l yang direpresentasikan dalam bentuk persamaan umum:

$$ax + by + c = 0$$

dan sebuah titik $P = (x_P, y_P)$. Tentukan posisi titik P terhadap garis l dengan cara mengimplementasikan fungsi `gradient` dan `CheckPointPosition` pada file header yang telah diberikan.

Input	Output
1 0 -3 5 0	Left
1 0 -3 1 7	Right
2 -1 -1 1 1	On Line

Table 1. Contoh input dan output soal 1 Modul 1

A. Source Code

line.cpp

1	#include "line.h"
2	using namespace std;
3	
4	double gradient(const Line * l, const Point * p){
5	return l->a * p->x + l->b * p->y + l->c;
6	}
7	
8	string CheckPointPosition(double gradient){
9	if(gradient > EPSILON){
10	return "Left";
11	}else if(gradient < -EPSILON){
12	return "Right";
13	}else{
14	return "On Line";
15	}
16	}

Table 2. Source Code line.cpp

prob1.cpp

1	#include "line.h"
2	#include <iostream>
3	using namespace std;
4	
5	int main() {

6	Line line1;
7	Point point1;
8	
9	cin >> line1.a >> line1.b >> line1.c;
10	cin >> point1.x >> point1.y;
11	
12	cout << CheckPointPosition(gradient(&line1,
13	&point1));
14	
15	return 0;
16	}

Table 3. Source Code prob1.cpp

B. Output Program

```

kimsora .../Algo&Data/struct_pointer v15.2.0 ▲ v3.31.6
> ./prob1.exe
1 0 -3 5 0
Left

kimsora .../Algo&Data/struct_pointer v15.2.0 ▲ v3.31.615s
> ./prob1.exe
1 0 -3 1 7
Right

kimsora .../Algo&Data/struct_pointer v15.2.0 ▲ v3.31.6
> ./prob1.exe
2 -1 -1 1 1
On Line

```

Gambar 1. Screenshot Hasil Jawaban Soal 1 Modul 1

C. Pembahasan

Pada line.cpp baris [4 - 6], syntax digunakan untuk melakukan override fungsi dari “line.h” yang disediakan untuk menghitung nilai dari gradient berdasarkan dari persamaan garis dan juga input user, nilai didapat dari memasukan input user kedalam persamaan. Pada line.cpp baris [8 – 16], terdapat kondisional digunakan untuk menentukan posisi garis terhadap titik, kanan artinya garis ada dikanan titik.

Pada prob1.cpp baris [6 – 13], syntax digunakan untuk membuat object line dan point yang atribut nya kemudian dimasukan oleh input user, dan kemudian memanggil fungsi dari line.h untuk menentukan posisi garis terhadap titik.

SOAL 2

Diberikan sebuah lingkaran C dengan pusat (c_x, c_y) dan jari-jari $r > 0$, serta sebuah titik $P = (p_x, p_y)$. Tentukan posisi titik P terhadap lingkaran C dan mengimplementasikan fungsi `distance` dan `CheckPointInCircle` pada file header yang diberikan.

Input	Output
0 0 5 3 4	On Circle
0 0 5 1 1	Inside
0 0 5 4 4	Outside
-2 -2 4 2 -2	On Circle

Table 4. Contoh input dan output soal 2 Modul 1

A. Source Code

circle.cpp

```
1  #include "circle.h"
2  #include <cmath>
3  using namespace std;
4
5  double distance(const Circle *c, const Point *p) {
6      return (pow(p->x - c->centre.x, 2) + pow(p->y - c-
7  >centre.y, 2)) /
8          pow(c->radius, 2);
9  }
10
11 string CheckPointInCircle(double distance) {
12     if (distance > 1 + EPSILON) {
13         return "Outside";
14     } else if (distance < 1 - EPSILON) {
15         return "Inside";
16     } else {
17         return "On Circle";
18     }
19 }
```

Table 5. Source Code circle.cpp

prob2.cpp

```
1  #include "circle.h"
2  #include <iostream>
3  using namespace std;
4
5  int main() {
```

6	Circle circle1;
7	Point point1;
8	
9	cin >> circle1.centre.x >> circle1.centre.y >> cir-
10	cle1.radius;
11	cin >> point1.x >> point1.y;
12	
13	cout << CheckPointInCircle(distance(&circle1,
14	&point1));
15	
16	return 0;
17	}

Table 6. Source Code prob2.cpp

B. Output Program

```

kimsora .../Algo&Data/struct_pointer v15.2.0 ▲ v3.31.6
> ./prob2.exe
0 0 5 3 4
On Circle

kimsora .../Algo&Data/struct_pointer v15.2.0 ▲ v3.31.67s
> ./prob2.exe
0 0 5 1 1
Inside

kimsora .../Algo&Data/struct_pointer v15.2.0 ▲ v3.31.67s
> ./prob2.exe
0 0 5 4 4
Outside

kimsora .../Algo&Data/struct_pointer v15.2.0 ▲ v3.31.6
> ./prob2.exe
-2 -2 4 2 -2
On Circle

```

Gambar 2. Screenshot Hasil Jawaban Soal 2 Modul 1

C. Pembahasan

Pada circle.cpp baris [5 - 9], syntax digunakan untuk melakukan override fungsi dari “circle.h” yang disediakan untuk menghitung nilai dari distance antara titik dan garis lingkaran berdasarkan dari persamaan garis dan juga input user, persamaan garis nya adalah $(x-a)^2 + (y-b)^2 = r^2$ atau $((x-a)^2 + (y-b)^2) / r^2$, nilai didapat dari memasukan input user kedalam persamaan. Pada circle.cpp baris [11 – 19], kita menggunakan persamaan kedua karena lebih mudah, yang artinya jika hasil nya 1.00 maka berada di garis begitu juga <1.00 di dalam dan >1.00 di luar.

Pada prob2.cpp baris [6 – 14], syntax digunakan untuk membuat object circle dan point yang atribut nya kemudian dimasukan oleh input user, dan kemudian memanggil fungsi dari circle.h untuk menentukan posisi garis terhadap titik.

MODUL 2: STACK DAN QUEUE

SOAL 1

Implementasi Stack

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

stack.cpp

```
1  #include "stack.h"
2
3  void init(Stack *s) {
4      s->top = new int;
5      *s->top = -1;
6  }
7  bool isEmpty(const Stack *s) { return *s->top <= -1; }
8  bool isFull(const Stack *s) { return *s->top >= MAX - 1; }
9  void push(Stack *s, int value) {
10     if (isFull(s)) {
11         throw "Stack Penuh";
12     }
13     *(s->top) += 1;
14     s->data[*s->top] = value;
15 }
16 void pop(Stack *s) {
17     if (isEmpty(s)) {
18         throw "Stack Kosong";
19     }
20     *(s->top) -= 1;
21 }
22 int peek(const Stack *s) {
23     if (isEmpty(s)) {
24         throw "Stack Kosong";
25     }
26     return s->data[*s->top];
27 }
28
```

B. Pembahasan

Pada soal ini ada 6 fungsi yang perlu di implementasikan ada 1 yang berfungsi seperti constructor, 3 sebagai getter yang hanya melakukan cek / return sesuatu, dan 2 fungsi utama operasi.

Alur Fungsi

- A. `init()` : Pertama fungsi akan melakukan inisialisasi untuk field `top` biar nilai nya bisa di set, Kedua fungsi ini akan memberikan nilai -1 sebagai nilai awal yang menandakan stack ini kosong.
- B. `isEmpty()` : Fungsi ini hanya melakukan cek apakah nilai dari `top` kurang atau sama dengan -1, untuk menandakan stack ini kosong.
- C. `isFull()` : Fungsi ini hanya melakukan cek apakah nilai dari `top` lebih atau sama dengan `MAX` dikurang 1 yang ada di header file.
- D. `peek()` : Pertama fungsi ini akan memanggil fungsi `isEmpty` untuk cek apakah stack kosong, jika kosong langsung throw kosong saja. Kedua fungsi ini akan mengembalikan nilai stack yang paling atas.
- E. `push()` : Pertama fungsi ini akan memanggil fungsi `isFull` untuk cek apakah stack penuh. Jika stack penuh akan throw penuh. Kedua fungsi ini akan menambah 1 nilai dari pointer `top` atau memajukan nya. Ketiga dia akan memasukan nilai baru kedalam posisi `top` yang baru.
- F. `pop()` : Pertama fungsi ini akan memanggil fungsi `isEmpty` untuk cek apakah stack kosong, jika kosong langsung throw kosong saja. Kedua fungsi ini akan mengurangi satu nilai dari `top`, atau menurunkan nilainya sehingga posisi `top` bergeser kebawah.

Kenapa Ini Bekerja?

`Init()` dibuat sebagai constructor dimana tugas utama dari constructor adalah melakukan inisialisasi dari sebuah object pada kasus ini adalah stack. Maka dari itu fields atau atribut nya yang pada header berbentuk pointer perlu di inisialisasi juga supaya memiliki alamat pointer, jika tidak pointer itu tidak akan memiliki alamat memory sehingga nilai nya tidak bisa dimasukan.

`IsEmpty` dibuat untuk cek apakah stack ini kosong, dan karena kondisi awal stack saat kosong `top` nya diset ke -1 maka kita hanya perlu cek apakah `top` stack saat ini kurang atau sama dengan -1.

isFull dibuat untuk cek apakah stack ini penuh, karena max sudah ada didefinisikan pada file header kita hanya perlu lakukan cek apakah top sama dengan max – 1 karena index nya mulai dari 0.

peek dibuat untuk mengembalikan nilai teratas / terbaru dari stack, untuk memastikan fungsi ini berjalan dengan sempurna kita perlu cek apakah stack kosong dulu, untuk mengembalikan nilai stack nya kita cukup panggil saja data nya dengan index top.

push dibuat untuk menambahkan data kedalam stack, sebelum fungsi ini bisa menambahkan data kedalam stack dia perlu cek apakah stack penuh dulu biar tidak terjadi overflow, fungsi ini akan increment nilai top baru memasukan data nya kedalam stack dengan index top, karena ada cek ini kita tidak bisa memasukan nilai yang lebih besar dari max.

pop dibuat untuk mengeluarkan data dari dalam stack, karena data dalam stack tidak perlu dihapus dan bisa di overwrite maka fungsi ini tidak perlu melakukan reset atau null kan data, tetapi hanya perlu menurunkan nilai top saja karena kita hanya perlu data yang ada dibawah top dan top saja, dan jika setiap pop data sebelumnya di hapus maka membuat fungsi ini jauh lebih lambat.

Analisis Kompleksitas

Semua fungsi yang ada pada stack ini memiliki kompleksitas ruang dan waktu $O(1)$, karena tidak ada loop atau pemanggilan fungsi yang recursive berulang sebanyak n dan hanya operasi standar tanpa ada struct yang berpotensi memiliki size n.

SOAL 2

Si Palui dan Mesin Hitung Aneh

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- A. sebuah bilangan bulat, atau
- B. sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- C. Mesin membaca simbol dari kiri ke kanan.
- D. Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- E. Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- A_i adalah nilai bilangan pada input
- Simbol yang valid hanya:
 - bilangan bulat
 - operator: +, -, *, /

Format Input

n

$A_1 A_2 \dots A_n$

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

Table 8. Contoh input dan output soal 2 Modul 2

A. Source Code

prob1.cpp

```

1 #include "stack.h"
2 #include <cmath>
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 // Postfix eval
8 int main() {
9     // init
10    int n;
11    Stack stack1;
12    init(&stack1);
13
14    // input n
15    cin >> n;
16    // loop input sebanyak n
17    for (int i = 0; i < n; i++) {
18        // container untuk input pake string karena campur symbol
19        string input;
20        cin >> input;
21
22        // cek apakah input digit atau jika size nya lebih dari 1
23        dan memiliki -
24        // intinya cek jika ini bukan symbol
25        if (isdigit(input[0]) || input.size() > 1 && input[0] ==
26        '-') {
27            // push ke stack
28            push(&stack1, stoi(input));
29        } else {
30            // jika bukan symbol keluarkan 2 angka
31            int kanan = peek(&stack1);

```

```

32     pop(&stack1);
33     int kiri = peek(&stack1);
34     pop(&stack1);
35
36     // lakukan proses sesuai dengan symbol
37     if (input == "+")
38         push(&stack1, kiri + kanan);
39     else if (input == "-")
40         push(&stack1, kiri - kanan);
41     else if (input == "*")
42         push(&stack1, kiri * kanan);
43     else if (input == "/")
44         push(&stack1, kiri / kanan);
45     else if (input == "^")
46         push(&stack1, (int)pow(kiri, kanan));
47     }
48 }
49 cout << peek(&stack1) << endl;
50 return 0;
51 }
52

```

Table 9. Source Code prob1.cpp

B. Output Program

```

kimsora .../module-2-stack-and-queue-biahhlil main [?]
> ./prob1
3
2 3 +
5

kimsora .../module-2-stack-and-queue-biahhlil main [?]
> ./prob1
9
5 1 2 + 4 * + 3 -
14

kimsora .../module-2-stack-and-queue-biahhlil main [?]
> ./prob1
5
10 2 / 3 *
15

kimsora .../module-2-stack-and-queue-biahhlil main [?]
> ./prob1
9
5 1 2 + 4 * + 3 -
14

```

Gambar 3. Screenshot Hasil Jawaban Soal 2 Modul 2

C. Pembahasan

File ini adalah main file dimana program utama berjalan.

Alur Algoritma

1. init : ini adalah bagian untuk menginisialisasi variable dan struct yang diperlukan, pada algoritma ini adalah variable `n`, `stack` `stack1`.
2. Input `n` : mengambil input `n` dari user.
3. Loop : disini kita melakukan loop sebanyak `n` kali untuk melakukan postfix eval.
4. User input : didalam loop ini inisialisasi variable bantuan “input” untuk menampung input user, dan kemudian mengambil input dari user untuk “input”.
5. Check if not symbol : masih didalam loop, kita lakukan cek apakah input user ini adalah angka bulat dan bukan symbol, jika cek ini benar maka masukan angka ke `stack1`
6. If symbol : jika symbol maka kita perlu mengeluarkan 2 angka terakhir dari `stack1` dengan cara inisialisasi 2 variable bantuan kanan dan kiri, masukan angka terbaru (atas) kedalam kanan dan lakukan pop pada `stack1`, dan kemudian masukan angka kedua (setelah atas) ke kiri dan lakukan pop pada `stack1`.
7. Process symbol : setelah kedua angka kanan dan kiri dikeluarkan maka algoritma melakukan cek symbol apa yang sesuai dengan input, dan lakukan operasi arithmatic nya masing-masing, dan hasil nya akan di push kedalam `stack1`.
8. Hal ini dilakukan terus sampai `n` kali.
9. Output : Print isi terakhir yang ada pada `stack1` yang merupakan hasil dari proses.

Kenapa Ini Bekerja?

Algoritma ini bekerja dengan cara mengambil `n` input dari user dan melakukan postfix eval sebanyak `n` kali. Karena sifat stack yang Last In, First Out (LIFO) membuatnya sangat cocok untuk postfix karena sifat ini memastikan symbol yang muncul akan selalu diproses terhadap 2 angkat teratas. Dan hasilnya akan disimpan kembali kedalam stack, yang kemudian proses nya bisa dilanjutkan, jika yang masuk setelah ini angka maka simpan, dan jika yang masuk symbol maka angka hasil proses sebelum nya dan yang dibawah nya akan kembali di proses, hingga hanya ada sisa satu angka yang ada pada stack.

Analisis Kompleksitas

Algoritma ini memiliki kompleksitas waktu dan ruang $O(n)$, karena algoritma ini memiliki 1 loop sebanyak `n` kali yang membuat waktu nya menjadi $O(n)$, dan algoritma ini juga membuat sebuah struktur data yang size nya sebesar `n` hal ini membuat komposisi ruang nya menjadi $O(n)$.

SOAL 3

Implementasi Queue

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

queue.cpp

```
1  #include "queue.h"
2
3  void init(Queue *q) {
4      q->front = new int;
5      q->rear = new int;
6      *q->front = -1;
7      *q->rear = -1;
8  }
9  bool isEmpty(const Queue *q) { return *q->front == -1; }
10 bool isFull(const Queue *q) { return (*q->rear + 1) % MAX ==
11 *q->front; }
12 void enqueue(Queue *q, int value) {
13     if (isFull(q)) {
14         throw "Queue Penuh";
15     }
16     if (isEmpty(q)) {
17         *q->front = 0;
18         *q->rear = 0;
19     } else {
20         *q->rear = (*q->rear + 1) % MAX;
21     }
22     q->data[*q->rear] = value;
23 }
24 void dequeue(Queue *q) {
25     if (isEmpty(q)) {
26         throw "Queue Kosong";
27     }
28     if (*q->front == *q->rear) {
29         *q->front = -1; // Reset ke setelan pabrik
30         *q->rear = -1;
31     } else {
32         *q->front = (*q->front + 1) % MAX;
```

```

33     // Jika masih banyak elemen, front maju melingkar
34     *q->front = (*q->front + 1) % MAX;
35 }
36 }
37 int front(const Queue *q) {
38     if (isEmpty(q)) {
39         throw "Queue Kosong";
40     }
41     return q->data[*q->front];
42 }
43 int back(const Queue *q) {
44     if (isEmpty(q)) {
45         throw "Queue Kosong";
46     }
47     return q->data[*q->rear];
48 }

```

Table 10. Source Code queue.cpp

B. Pembahasan

Pada soal ini ada 7 fungsi, dimana 1 fungsi constructor, 4 getter, dan 2 fungsi utama.

Alur Fungsi

1. Init() : hampir sama dengan di soal 1 yang membedakan disini inisialisasi 2 field yaitu front dan rear.
2. isEmpty() : Fungsi ini melakukan cek apakah queue kosong dengan cara cek apakah front sama dengan -1.
3. isFull() : Fungsi ini melakukan cek apakah queue penuh dengan cara membandingkan apakah rear + 1 mod Max sama dengan front, dimana ini cek apakah queue berikutnya memiliki posisi sama dengan front menandakan queue penuh dan tidak ada posisi kosong.
4. front() : Fungsi ini pertama memanggil isEmpty untuk cek apakah queue kosong, jika kosong throw queue kosong. Kedua return data queue dengan index front.
5. back() : Fungsi ini pertama memanggil isEmpty untuk cek apakah queue kosong, jika kosong throw queue kosong. Kedua return data queue dengan index rear.
6. enqueue() : Pertama fungsi ini memanggil isFull, jika full throw queue penuh. Kedua fungsi ini memanggil isEmpty, jika kosong fungsi ini akan mengubah front dan rear menjadi 0 biar siap untuk di masukan data, jika tidak kosong fungsi ini akan memajukan rear dengan rear + 1 mod Max, supaya jika rear sudah posisi max maka akan kembali ke 0 lagi untuk mengisi posisi yang kosong.
7. dequeue() : Pertama fungsi ini memanggil isEmpty untuk cek apakah queue kosong, jika kosong throw queue kosong. Kedua fungsi ini melakukan cek apakah front dan

rear memiliki posisi yang sama, reset front dan rear menjadi -1 untuk menandakan queue kosong, jika tidak $\text{front} + 1 \bmod \text{max}$ artinya front akan maju satu posisi dan jika sudah mencapai max front akan kembali ke posisi 0.

Kenapa Ini Bekerja?

Init sama seperti soal 1 hanya saja bedanya disini ada 2 field yang perlu di inialisasi.

IsEmpty adalah fungsi getter yang bertugas untuk cek apakah queue kosong, fungsi melakukan cek apakah queue berada pada kondisi default atau kosong nya yang ditandai dengan front dan rear memiliki nilai -1, ini diset pada init dan saat dequeue menemukan front dan rear nya pada posisi yang sama artinya queue kosong dia yang akan set front dan rear menjadi -1

isFull adalah fungsi getter yang bertugas untuk cek apakah queue penuh, fungsi ini melakukan cek dengan cara membandingkan apakah posisi $\text{rear} + 1 \bmod \text{max}$ (posisi rear setelah ini) sama dengan front, yang menandakan posisi rear setelah ini sama dengan front dan ini indikasi bahwa queue penuh. Karena rear maju ketemu front itu artinya penuh.

Front adalah fungsi getter yang bertugas untuk mengembalikan data yang ada pada front, fungsi ini melakukannya dengan mengembalikan data pada posisi front. Fungsi ini melakukan cek isEmpty untuk memastikan queue tidak kosong.

Rear adalah fungsi getter yang bertugas untuk mengembalikan data yang ada pada rear, hal ini dilakukan dengan mengembalikan data yang ada pada posisi rear. Fungsi ini melakukan cek isEmpty untuk memastikan queue tidak kosong.

Enqueue fungsi ini adalah fungsi utama yang bertugas untuk memasukan data kedalam queue, sebelum memasukan data fungsi ini melakukan cek isFull untuk memastikan queue tidak penuh dan data bisa dimasukan, kemudian fungsi ini melakukan cek isEmpty, hal ini dilakukan untuk memastikan queue sudah siap dan field nya tidak kosong, jika kosong fungsi ini akan menyiapkan queue dengan mengubah front dan rear menjadi 0, jika queue tidak kosong maka fungsi akan memajukan posisi rear tentu saja dengan mod Max, dan kemudian fungsi akan memasukan data ke posisi rear. Bisa terlihat fungsi ini cukup ketat dengan pengecekan nya, hal ini untuk mencegah ada nya overlap antara data yang ada di rear dan front.

Dequeue fungsi adalah fungsi utama yang bertugas untuk mengeluarkan data dari queue yaitu data di front, sebelum fungsi ini memajukan front fungsi ini melakukan isEmpty, dan melakukan cek apakah posisi front dan rear sama, jika sama artinya queue ini sisa satu posisi dan kita akan reset kondisinya ke -1 yaitu kosong, jika tidak maka fungsi ini akan memajukan posisi front tentu saja dengan mod Max supaya posisi nya bisa kembali ke 0 jika sudah max. fungsi ini lah yang bekerja untuk menjaga status queue karena dia yang bisa melakukan reset queue menjadi kosong kembali

Analisis Kompleksitas

Semua fungsi yang ada pada stack ini memiliki kompleksitas ruang dan waktu $O(1)$, karena tidak ada loop atau pemanggilan fungsi yang recursive berulang sebanyak n dan hanya operasi standar tanpa ada struct yang berpotensi memiliki size n .

SOAL 4

Si Palui dan Mesin Hitung Aneh

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n : jumlah elemen
- k : ukuran jendela
- A_i : elemen array

Format Input

n k

A_1 A_2 ... A_n

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2	3 1 -1 2 6

A. Source Code

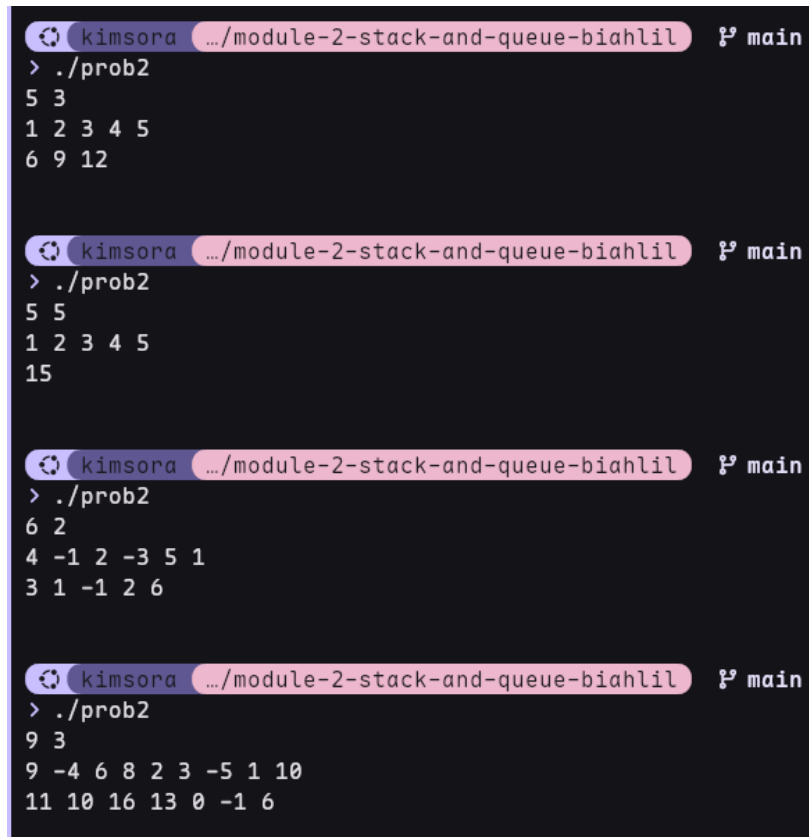
prob2.cpp

```

1  #include "queue.h"
2  #include <iostream>
3  using namespace std;
4
5  // sliding window queue
6  int main() {
7      // init
8      int n;
9      int k;
10     int total = 0;
11     Queue queue1;
12     init(&queue1);
13
14     // input n dan k (n = banyak input, k = window yang ingin
15     dijumlahkan)
16     cin >> n >> k;
17     for (int i = 0; i < n; i++) {
18         // container input angka
19         int input;
20         cin >> input;
21         // lanjut isi jika queue nya belum penuh alias belum
22         sepanjang k
23         if (i < k) {
24             enqueue(&queue1, input);
25             total += input;
26             // jika sudah sepanjang k print total
27             if (i == k - 1) {
28                 cout << total;
29             }
30         } else {
31             // untuk lebih akan pake sliding window
32             int yang_dibuang = front(&queue1);
33             dequeue(&queue1);
34             enqueue(&queue1, input);
35             // total cuma mengurangi nilai yang dibuang alias
36             front dna ditambah
37             // nilai baru alias rear baru.
38             total = total - yang_dibuang + input;
39             cout << " " << total;
40         }
41     }
42     cout << endl;
43     return 0;

```

B. Output Program



```

kimsora ~/module-2-stack-and-queue-biahlil main
> ./prob2
5 3
1 2 3 4 5
6 9 12

kimsora ~/module-2-stack-and-queue-biahlil main
> ./prob2
5 5
1 2 3 4 5
15

kimsora ~/module-2-stack-and-queue-biahlil main
> ./prob2
6 2
4 -1 2 -3 5 1
3 1 -1 2 6

kimsora ~/module-2-stack-and-queue-biahlil main
> ./prob2
9 3
9 -4 6 8 2 3 -5 1 10
11 10 16 13 0 -1 6

```

Gambar 4. Screenshot Hasil Jawaban Soal 4 Modul 2

C. Pembahasan

File ini adalah main file dimana program utama berjalan.

Alur Algoritma

1. init : ini adalah bagian untuk menginisialisasi variable dan struct yang diperlukan, pada algoritma ini adalah variable n k total, queue queue1.
2. Input n dan k : mengambil input n dan k dari user.
3. Loop : disini kita melakukan loop sebanyak n kali untuk melakukan sliding window sum.
4. User input : didalam loop ini inialisasi variable bantuan "input" untuk menampung input user, dan kemudian mengambil input dari user untuk "input".

5. Check if $i < k$: masih didalam loop ini algoritma akan melakukan cek apakah k atau window sudah penuh, jika belum maka input user akan di enqueue. Dan kemudian tambahkan nilai input user kedalam total. Masih didalam cek window dilakukan cek $i == k - 1$ jika input sudah memenuhi window maka bisa print total.
6. Lebihan sliding window : jika check window sudah penuh maka kita perlu inisialisasi variable bantuan yang_dibuang yang memiliki value dari data yang berada pada posisi front queue. Kemudian algoritma akan melakukan dequeue yang dilanjutkan dengan enqueue input terbaru user.
7. Total baru : setelah dequeue dan enqueue dilakukan maka algoritma akan memperbarui nilai dari total dengan dikurangi oleh yang_dibuang dan ditambah oleh input terbaru. Dan kemudian total akan di print.
8. Hal ini dilakukan terus sampai n kali.

Kenapa Ini Bekerja?

Algoritma ini bekerja dengan cara mengambil input n dan k dari user, dan melakukan proses sliding window untuk mengambil total nilai dari setiap window. Sifat First in First out dari queue sangat cocok untuk menyelesaikan masalah pada soal ini, dan keperluan untuk melakukan operasi untuk beberapa member queue saja membuat algoritma sliding window ini menjadi solusi yang pass. Algoritma ini bekerja dengan cara mengisi window / member yang ingin diproses, dan jika sudah penuh baru lakukan proses, ini dilakukan seterusnya seiring dengan member lama dikeluarkan dan member baru dimasukan.

Analisis Kompleksitas

Algoritma ini memiliki kompleksitas waktu $O(n)$ dan Kompleksitas Ruang $O(k)$, karena algoritma ini memiliki 1 loop sebanyak n kali yang membuat waktu nya menjadi $O(n)$, dan untuk kompleksitas ruang, nilainya adalah $O(k)$, karena meskipun total data ada sebanyak n, struktur data Queue pada algortima ini hanya akan menyimpan elemen maksimal sebanyak kapasitas jendela (k) di memori secara bersamaan (karena elemen lama langsung dikeluarkan).

MODUL 3: LINKED LIST

SOAL 1

Implementasi Senarai Berantai Tunggal Melingkar

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

`single_linked_list.cpp`

```
1 #include "single_linked_list.h"
2 #include <iostream>
3 #include <stdexcept>
4
5 void SingleLinkedList::init() {
6     head = nullptr;
7     tail = nullptr;
8     size = 0;
9 }
10
11 bool SingleLinkedList::is_empty() { return size == 0; }
12
13 void SingleLinkedList::add_front(int val) {
14     Node *new_node = new Node();
15     new_node->data = val;
16
17     if (is_empty()) {
18         new_node->next = new_node;
19         head = new_node;
20         tail = new_node;
21     } else {
22         new_node->next = tail->next;
23         tail->next = new_node;
24         head = new_node;
25     }
26     size++;
```

```

27 }
28
30 void SingleLinkedList::add_back(int val) {
31     Node *new_node = new Node();
32     new_node->data = val;
33
34     if (is_empty()) {
35         new_node->next = new_node;
36         head = new_node;
37         tail = new_node;
38     } else {
39         new_node->next = tail->next;
40         tail->next = new_node;
41         tail = new_node;
42     }
43     size++;
44 }
45
46 void SingleLinkedList::add_idx(int val, int idx) {
47     if (idx < 0 || idx > size) {
48         // size pake > karena zero based index size 99 posisi nya
50 98
51         // dan yang diminta disini jika param idx artinya posisi
52 idx bukan idx - 1
53         // misal idx 5 berarti diminta posisi 5 (0,1,2,3,4,5)
54 bukan 4
55         throw std::out_of_range("Index node melebihi size saat
56 ini");
57     } else if (idx == 0) {
58         add_front(val);
59     } else if (idx == size) {
60         add_back(val);
61     } else {
62         Node *new_node = new Node();
63         new_node->data = val;
64
65         Node *current = head;
66         for (int i = 0; i < idx - 1; i++) {
67             current = current->next;
68         }
69         new_node->next = current->next;
70         current->next = new_node;
71         size++;
72     }
73 }
74
75 void SingleLinkedList::delete_front() {
76     if (is_empty()) {
77         throw std::logic_error("List Kosong");
78     } else if (size == 1) {
79         delete head;
80         head = nullptr;

```

```

81     tail = nullptr;
82 } else {
83     Node *old_head = head;
84     head = head->next;
85     tail->next = head;
86     delete old_head;
87 }
88 size--;
89 }
90
91 void SingleLinkedList::delete_back() {
92     if (is_empty()) {
93         throw std::logic_error("List Kosong");
94     }
95     if (size == 1) {
96         delete head;
97         head = nullptr;
98         tail = nullptr;
99     } else {
100         Node *current = head;
101         while (current->next != tail) {
102             current = current->next;
103         }
104         current->next = head;
105         delete tail;
106         tail = current;
107     }
108     size--;
109 }
110
111 void SingleLinkedList::delete_idx(int idx) {
112     if (is_empty()) {
113         throw std::logic_error("List Kosong");
114     } else if (idx < 0 || idx >= size) { // size pake >= karena
115         zero based index                // size 99 posisi nya
116
117         98
118         throw std::out_of_range("Index node melebihi size saat
119         ini");
120     } else if (idx == 0) {
121         delete_front();
122     } else if (idx == size - 1) {
123         delete_back();
124     } else {
125         Node *current = head;
126         for (int i = 0; i < idx - 1; i++) {
127             current = current->next;
128         }
129         Node *to_delete = current->next;
130         current->next = to_delete->next;
131         delete to_delete;
132         size--;

```

```

133     }
134 }
135
136 void SingleLinkedList::display() {
137     if (is_empty()) {
138         std::cout << "List Kosong\n";
139     } else {
140         Node *current = head;
141         for (int i = 0; i < size; i++) {
142             std::cout << current->data << " -> ";
143             current = current->next;
144         }
145         std::cout << "Kembali Ke Head (Head Data : " << head->data
146 << ")\n";
147     }
148 }
149
150 int SingleLinkedList::get(int idx) {
151     if (is_empty()) {
152         throw std::logic_error("List Kosong");
153     } else if (idx < 0 || idx >= size) {
154         throw std::out_of_range("Index node melebihi size saat
155 ini");
156     } else {
157         Node *current = head;
158         for (int i = 0; i < idx; i++) {
159             current = current->next;
160         }
161         return current->data;
162     }
163 }
164
165 void SingleLinkedList::set(int val, int idx) {
166     if (is_empty()) {
167         throw std::logic_error("List Kosong");
168     } else if (idx < 0 || idx >= size) {
169         throw std::out_of_range("Index node melebihi size saat
170 ini");
171     } else {
172         Node *current = head;
173         for (int i = 0; i < idx; i++) {
174             current = current->next;
175         }
176         current->data = val;
177     }
178 }
179
180 void SingleLinkedList::clear() {
181     while (!is_empty()) {
182         delete_front();
183     }
184 }

```

B. Pembahasan

Alur Fungsi

- Penambahan di ujung (`add_front` dan `add_back`): Pertama memori dinamis akan dialokasikan pada heap untuk node baru. Node baru kemudian diisi dengan nilai data. Kemudian cek jika list masih kosong maka node akan menunjuk dirinya sendiri dan menjadi head sekaligus tail. Jika tidak kosong penunjuk next dari node baru dibuat menunjuk head lama, kemudian `tail → next` menunjuk node baru. Disini perbedaan pada `add_front` dan `add_back`, untuk front head dipindah ke node baru sedangkan back tail yang dipindah ke node baru.
- Penambahan node di posisi yang bebas: program disini memvalidasi batas index, jika valid pointer bantuan akan berjalan (`traverse`) mulai dari head dan berhenti di `nodeindex - 1`. kemudian node baru next akan menunjuk node sebelumnya, lalu node sebelum mengubah next nya untuk menunjuk node baru.
- Penghapusan node (`delete_front`, `delete_back`, `delete_idx`):
 - front: head digeser ke node didepannya, kemudian `tail → next` diubah menunjuk ke head baru. Node head lama dihapus (`delete`).
 - Back dan idx: buat pointer bantuan, pointer ini akan berjalan ke node sebelum target index yang ingin dihapus. Next dari node sebelum target dialihkan agar melewati target menunjuk node setelah target. Setelah list aman node target akan dihancurkan dari memori.
- Pembersih memori / list (`clear`): fungsi ini melakukan perulangan while dengan kondisi `!is_empty()` yang memanggil `delete_front()` secara terus menerus. Karena fungsi penghapus sudah akan memanggil `delete`, dan menghapus list.

Kenapa Ini Bekerja?

Kenapa `add_front` dan `add_back` mirip bedanya hanya pada penempatan head dan tail. Hal ini karena struktur linked list yang melingkar, penambahan di depan atau belakang menempati titik yang sama yaitu antara head dan tail.

Karena setiap fungsi `delete` disini sudah memanggil `delete` diakhir maka ini sudah mencegah memory leak dan karena itu `clear` hanya perlu memanggil fungsi `delete` saja untuk mengosongkan list.

Pada `add` atau `delete` di `idx` kenapa kita menggunakan validasi yang sedikit berbeda? Karena untuk front kita bisa memasukan node pada posisi `idx > size` karena ini zero based jadi jika `idx 4` itu size nya 5 dan kita ingin masukan di posisi/`idx 5` maka kita hanya perlu pang-

giladd_back, berbeda dengan delete idx kita memastikan posisi $idx \geq size$ disini mengisaratkan bahwa kita tidak bisa menghapus idx 5 jika size cuman 5 karena size lima posisi paling akhir adalah 4.

Analisis Kompleksitas

Kompleksitas waktu: $O(1)$ untuk add_front, add_back, delete_front. Tetapi $O(n)$ untuk add_idx, delete_idx, get, dan set karena fungsi-fungsi ini memerlukan traversal (looping) dari head ke indeks tujuan (n).

Kompleksitas Ruang: $O(1)$ memori untuk tiap fungsi nya, tetapi karena ini adalah struktur data maka secara keseluruhan Struktur data linked list ini memiliki $O(n)$ kompleksitas.

SOAL 2

Si Palui dan Relikui Siklus Resonansi

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh N buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh K langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ($K = K + 1$).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ($K = K - 1$).
- Jika pada suatu titik energi lompatan habis atau negatif ($K \leq 0$), sistem akan mengalami reset dan mengembalikan nilai lompatan ke nilai K awal (saat pancaran energi pertama kali diinisialisasi).

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

Batasan

- $1 \leq N \leq 104$
- $1 \leq K \leq 109$
- $1 \leq A_i \leq 106$ (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari Soal 1.

Format Input

N K

A_1 A_2

...

A_N

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa.

Input	Output
5 2 1 4 3 6 2	6
4 5 10 11 12 13	12

Table 14. Contoh input dan output soal 2 Modul 3

A. Source Code

single_linked_list.cpp

```

1  #include "single_linked_list.h"
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      // Input n k
7      int N;
8      long long K; // long long karena K bisa sampai 10^9
9      cin >> N >> K;
10     long long initial_K = K;
11     // init
12     SingleLinkedList relikui;
13     relikui.init();
14     // Masukkan semua batu ke dalam list
15     for (int i = 0; i < N; i++) {
16         int batu;
17         cin >> batu;
18         relikui.add_back(batu);
19     }
20     // Mulai dari batu pertama
21     long long pos = 0;
22     // Loop selama batu yang tersisa lebih dari 1
23     while (relikui.size > 1) {
24         // Hitung lompatan
25         long long lompatan_asli = (K - 1) % relikui.size;
26         pos = (pos + lompatan_asli) % relikui.size;
27         // Ambil nilai ukiran batu tersebut
28         int nilai_batu = relikui.get(pos);
29         // Hancurkan batunya!
30         relikui.delete_idx(pos);
31         // batu "selanjutnya" otomatis menempati indeks 'pos' yang
32         sekarang!
33         pos = pos % relikui.size;
34     }

```

```

35 // cek anomali
36 if (nilai_batu % 2 == 0) {
37     K = K + 1;
38 } else {
39     K = K - 1;
40 }
41 // Aturan Reset K
42 if (K <= 0) {
43     K = initial_K;
44 }
45 }
46 // Cetak 1 batu yang tersisa
47 cout << relikui.get(0) << endl;
48 return 0;
50 }

```

Table 15. Source Code prob_a.cpp

B. Output Program

```

kimsora .../linked-list 1 main [?] v15.2.0
> ./prob_a
5 2
1 4 3 6 2
6

kimsora .../linked-list 1 main [?] v15.2.03s
> ./prob_a
4 5
10 11 12 13
12

kimsora .../linked-list 1 main [?] v15.2.04s
> ./prob_a
7 57
492 351 542 521 877 544 463
351

```

Gambar 5. Screenshot Hasil Jawaban Soal 2 Modul 3

C. Pembahasan

File ini adalah main file dimana program utama berjalan.

Alur Fungsi

- Inisialisasi data: memasukan semua angka uiran batu secara berurutan ke bagian belakang list dengan fungsi `add_back`.
- Loop Eksekusi: Loop while yang terus berjalan selama jumlah batu di dimensi masih lebih dari 1.
- Kalkulasi lompatan: menghitung jarak lompatan, untuk menghindari infinite loop saat angka K yang sangat besar, kita menggunakan matematika modulo. Posisi batu target dicari dengan rumus $pos = (pos + K + 1) \% size$.
- Penghancuran: nilai batu pada posisi target dibaca lalu batu tersebut dihancurkan dengan `delete_idx`. Setelah batu hancur batu selanjutnya otomatis bergeser merapat ke kiri sendiri.
- Anomali Energi: kita melakukan cek nilai batu yang akan dihancurkan. Jika genap nilai lompatan K ditambah 1. jika ganjil lompatan K dikurangi 1. Jika K turun hingga 0 atau negatif K akan koreset ke nilai awalnya.
- Pencetak: ketika while loop selesai program akan memanggil fungsi `get(0)` untuk mencetak hasil akhir.

Kenapa Ini Bekerja?

Algoritma Bekerja bagus karena menerapkan sifat melingkar (Circular) dari posisi formasi batu yang sebenarnya. Penggunaan rumus Modulo ($\% size$) pada langkah ke-3 adalah inti dari efisiensi dan keamanan program ini. Sesuai dengan batasan masalah (Constraint), nilai awal K dapat mencapai skala satu miliar (10⁹). Jika hanya melakukan pointer traversal (berjalan satu per satu menggunakan `current = current → next`) sebanyak K yang besar akan memicu kesalahan Constraint Time Limit Exceeded (TLE). Modulo menjadi solusi yang tepat untuk mendapatkan titik koordinat batu secara instan tanpa harus melakukan pointer traversal yang berlebihan. Kemudian `delete_idx` yang digunakan sebagai penghapus menggunakan delete untuk menjaga batas memori Constraint.

Analisis Kompleksitas

Kompleksitas Waktu: $O(n^2)$, karena terdapat loop utama sebanyak N kali untuk menghancurkan batu (while). Dan dilamanya `delete_idx` dipanggil yang dimana dia akan traverse sebanyak idx. Yang akhirnya akan menjadi $O(n^2)$.

Kompleksitas Ruang: $O(n)$, karena menyimpan N buah batu di dalam single linked list.

SOAL 3

Implementasi Senarai Berantai Ganda Melingkar

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList` beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-ONotation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

`double_linked_list.cpp`

```
1  #include "double_linked_list.h"
2  #include <iostream>
3  #include <stdexcept>
4
5  void DoubleLinkedList::init() {
6      head = nullptr;
7      tail = nullptr;
8      size = 0;
9  }
10
11 bool DoubleLinkedList::is_empty() { return size == 0; }
12
13 void DoubleLinkedList::add_front(char val) {
14     Node *new_node = new Node();
15     new_node->data = val;
16     if (is_empty()) {
17         new_node->next = new_node->prev = new_node;
18         head = new_node;
19         tail = new_node;
20     } else {
21         new_node->next = head;
22         new_node->prev = tail;
23         head->prev = new_node;
24         tail->next = new_node;
25         head = new_node;
26     }
27     size++;
28 }
29
30 void DoubleLinkedList::add_back(char val) {
```

```

31 Node *new_node = new Node();
32 new_node->data = val;
33 if (is_empty()) {
34     new_node->next = new_node->prev = new_node;
35     head = new_node;
36     tail = new_node;
37 } else {
38     new_node->next = head;
39     new_node->prev = tail;
40     tail->next = new_node;
41     head->prev = new_node;
42     tail = new_node;
43 }
44 size++;
45 }
46 void DoubleLinkedList::add_idx(char val, int idx) {
47     if (idx < 0 || idx > size) {
48         // size pake > karena zero based index size 99 posisi nya
50 98
51         // dan yang diminta disini jika param idx artinya posisi
52 idx bukan idx - 1
53         // misal idx 5 berarti diminta posisi 5 (0,1,2,3,4,5)
54 bukan 4
55         throw std::out_of_range("Index node melebihi size saat
56 ini");
57     } else if (idx == 0) {
58         add_front(val);
59     } else if (idx == size) {
60         add_back(val);
61     } else {
62         Node *new_node = new Node();
63         new_node->data = val;
64         Node *current = head;
65         for (int i = 0; i < idx - 1; i++) {
66             current = current->next;
67         }
68         new_node->next = current->next;
69         new_node->prev = current;
70         current->next->prev = new_node;
71         current->next = new_node;
72         size++;
73     }
74 }
75
76 void DoubleLinkedList::delete_front() {
77     if (is_empty()) {
78         throw std::logic_error("List Kosong");
79     } else if (size == 1) {
80         delete head;
81         head = nullptr;
82         tail = nullptr;
83     } else {

```

```

84     Node *old_head = head;
85     tail->next = head->next;
86     head->next->prev = tail;
87     head = head->next;
88     delete old_head;
89 }
90 size--;
91 }
92 void DoubleLinkedList::delete_back() {
93     if (is_empty()) {
94         throw std::logic_error("List Kosong");
95     }
96     if (size == 1) {
97         delete head;
98         head = nullptr;
99         tail = nullptr;
100     } else {
101         Node *old_tail = tail;
102         head->prev = tail->prev;
103         tail->prev->next = head;
104         tail = tail->prev;
105         delete old_tail;
106     }
107     size--;
108 }
109 void DoubleLinkedList::delete_idx(int idx) {
110     if (is_empty()) {
111         throw std::logic_error("List Kosong");
112     } else if (idx < 0 || idx >= size) { // size pake >= karena
113         zero based index                // size 99 posisi nya
114                                         //
115         98
116         throw std::out_of_range("Index node melebihi size saat
117         ini");
118     } else if (idx == 0) {
119         delete_front();
120     } else if (idx == size - 1) {
121         delete_back();
122     } else {
123         Node *current = head;
124         for (int i = 0; i < idx; i++) {
125             current = current->next;
126         }
127         current->prev->next = current->next;
128         current->next->prev = current->prev;
129         delete current;
130         size--;
131     }
132 }
133
134 void DoubleLinkedList::display() {
135     if (is_empty()) {

```



```

136     std::cout << "EMPTY\n";
137 } else {
138     Node *current = head;
139     for (int i = 0; i < size; i++) {
140         std::cout << current->data;
141         current = current->next;
142     }
143     std::cout << "\n";
144 }
145 }
146 char DoubleLinkedList::get(int idx) {
147     if (is_empty()) {
148         throw std::logic_error("List Kosong");
149     } else if (idx < 0 || idx >= size) {
150         throw std::out_of_range("Index node melebihi size saat
151 ini");
152     } else {
153         Node *current = head;
154         for (int i = 0; i < idx; i++) {
155             current = current->next;
156         }
157         return current->data;
158     }
159 }
160 void DoubleLinkedList::set(char val, int idx) {
161     if (is_empty()) {
162         throw std::logic_error("List Kosong");
163     } else if (idx < 0 || idx >= size) {
164         throw std::out_of_range("Index node melebihi size saat
165 ini");
166     } else {
167         Node *current = head;
168         for (int i = 0; i < idx; i++) {
169             current = current->next;
170         }
171         current->data = val;
172     }
173 }
174
175 void DoubleLinkedList::clear() {
176     while (!is_empty()) {
177         delete_front();
178     }
179 }

```

Table 16. Source Code double_linked_list.cpp

B. Pembahasan

Pada soal ini ada 7 fungsi, dimana 1 fungsi constructor, 4 getter, dan 2 fungsi utama.

Alur Fungsi

- Penambahan di ujung (`add_front` dan `add_back`): sama dengan yang circular single-linked list, memori dialokasikan untuk node baru. Namun sekarang ada 2 petunjuk arah (`next`, `prev`) untuk setiap node. Node baru diposisikan antara tail dan head. Node-baru `next` menunjuk head dan node baru `prev` menunjuk tail. Kemudian head `prev` menunjuk node baru dan tail `next` menunjuk node baru juga. Sama dengan single-perbedaan front dan back disini hanya pada proses akhir yang sama.
- Penambahan tengah list (`add_idx`): pointer berjalan ke lokasi `idx`. Setelah sampai node baru mengikat node setelahnya (`next`), dan juga node sebelumnya (`prev`). Setelah itu, penunjuk dari node setelahnya (`current next prev`) dan node sebelumnya (`current next`) diganti agar menunjuk node baru.
- Penghapusan di ujung (`delete_front` dan `back`): untuk front cukup simple kita cukup membuat node bantuan untuk menyimpan head dan mengubah ikatan head `next prev` untuk menunjuk tail, dan tail `next` menunjuk head `next`. Setelah itu head menjadi head `next`, baru node bantuan yang menyimpan head lama dihapus. Untuk back nya samasaja cuman beda nya ya ini tail yang dihapus.
- Penghapusan di tengah (`delete_idx`): karena adanya ikatan mundur (`prev`), kita bisa langsung saja ke index tujuan tidak perlu berhenti di `idx - 1`. setelah itu node sebelahkiri target (`current prev`) dan node kanan (`current next`) dibuat untuk saling mengikat secara langsung dan tidak melalui node target. Setelah node dikeluarkan dari list node bisa dihancurkan.

Kenapa Ini Bekerja?

Pada `delete_idx` kita perlu untuk membuat node yang bersebelahan dari target untuk saling berkaitan dulu untuk mengeluarkan target dari list, baru node target bisa di delete, jika deleted dilakukan deluan maka list akan mengalami dangling pointer dan program bisa crash.

Dengan adanya dua arah pointer yang menjadi keunggulan utama struktur ini bisa dirasakan pada `delete_idx` bagian setelah node target ditemukan koneksi antara node tetangga kiri dan kanan langsung bisa dibuat karena akses langsung ke `current prev` dan `current next`. Ini mengurangi pointer temporary yang dibutuhkan dan mengurangi kesalahan logika.

Analisis Kompleksitas

Semua fungsi yang ada pada stack ini memiliki kompleksitas ruang dan waktu $O(1)$, karena tidak ada loop atau pemanggilan fungsi yang recursive berulang sebanyak n dan hanya operasi standar tanpa ada struct yang berpotensi memiliki size n .

SOAL 4

Si Palui dan Tarik Tambang Dimensi

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi N karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari head, bergerak maju menggunakan next) dan Beta (dimulai dari tail, bergerak mundur menggunakan prev).

Palui melakukan observasi selama R ronde. Pada setiap ronde:

1. Proyektor Alpha dipaksa melompat maju sebanyak X langkah. Proyektor Beta dipaksa melompat mundur sebanyak Y langkah (simultan).
2. Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama-persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter next dari karakter yang hancur, dan Beta berpindah ke karakter prev dari karakter yang hancur.
3. Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antarakedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah R ronde selesai dieksekusi.

Batasan

- $\leq N \leq 5000$
- $1 \leq R \leq 5000$
- $1 \leq X, Y \leq 10^{18}$
- Struktur wajib menggunakan implementasi struktur data dari Soal 3.

Format Input

N R

C1 C2 ... CN

X1 Y1

X2 Y2

...

XR YR

Keterangan: C_i adalah satu karakter char tunggal.

Format Output

Karakter yang tersisa, dicetak berurutan mulai dari head hingga ke elemen terakhir, tanpa-pasi. Jika dimensi menjadi kosong, cetak EMPTY.

Input	Output
4 2 A B C D 1 1 2 3	ACB
5 3 V W X Y Z 100000 100000 2 2 5 5	VWYZ

Table 17. Contoh input dan output soal 4 Modul 3

A. Source Code

prob_b.cpp

```

1  #include "double_linked_list.h"
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      // Input N R
7      int N, R;
8      cin >> N >> R;
9      // Init dll Tambang
10     DoubleLinkedList tambang;
11     tambang.init();
12     // Loop input node char
13     for (int i = 0; i < N; i++) {
14         char c;
15         cin >> c;
16         tambang.add_back(c);
17     }
18     // Penandaan bahwa alpha awalnya head dan beta awalnya
19     tail
20     long long pos_alpha = 0;           // Index Head
21     long long pos_beta = tambang.size - 1; // Index Tail

```

```

22 // Main Loop / Process (input loncatan dan singular event)
23 for (int i = 0; i < R; i++) {
24     // Max 10^18 tidak bisa pake int tapi pake long long /
25     int64
26         long long X, Y;
27         cin >> X >> Y;
28         // Cek Jika N == 0 (semua karakter sudah hancur) hentikan
30     loop
31         if (tambang.size == 0)
32             break;
33         // Optimasi Lompatan
34         pos_alpha = (pos_alpha + X) % tambang.size;
35         long long langkah_beta = Y % tambang.size;
36         pos_beta = (pos_beta - langkah_beta + tambang.size) %
37         tambang.size;
38         char isi_alpha = tambang.get(pos_alpha);
39         char isi_beta = tambang.get(pos_beta);
40         // Cek Tabrakan atau Resonansi
41         if (pos_alpha == pos_beta) {
42             // panggil fungsi dari DoubleLinkedList
43             tambang.delete_idx(pos_alpha);
44             // Pastikan tidak melakukan Modulo 0 jika semua karakter
45             baru saja hancur
46             if (tambang.size > 0) {
47                 // Alpha tetap di posisinya secara angka, tapi karena
48                 size berkurang,
50                 // pastikan dia tidak out of bounds.
51                 pos_alpha = pos_alpha % tambang.size;
52                 // Beta mundur 1. Gunakan + size lalu Modulo agar aman
53                 dari angka
54                 // negatif
55                 pos_beta = (pos_beta - 1 + tambang.size) %
56                 tambang.size;
57             }
58             // PROBLEM Awalnya saya ubah menjadi circular (pos_alpha
59             + 1) %
60             // tambang.size == pos_beta || (pos_beta + 1) %
61             tambang.size == pos_alpha
62             // tetepi karena tidak benar semua saya coba kembali
63             pake yang biasa eh
64             // malah 100
65             } else if (pos_alpha + 1 == pos_beta || pos_beta + 1 ==
66             pos_alpha) {
67                 // Resonansi
68                 // Tukar karakternya menggunakan set()
69                 tambang.set(isi_beta, pos_alpha);
70                 tambang.set(isi_alpha, pos_beta);
71             }
72         }
73         tambang.display();
74         return 0;
75     }

```

B. Output Program

```

kimsora .../linked-list main [?] v15.2.022s
> ./prob_b
4 2
A B C D
1 1
2 3
ACB

kimsora .../linked-list main [?] v15.2.08s
> ./prob_b
5 3
V W X Y Z
100000 100000
2 2
5 5
VWYZ

```

Gambar 6. Screenshot Hasil Jawaban Soal 4 Modul 3

C. Pembahasan

File ini adalah main file dimana program utama berjalan.

Alur Fungsi

- Inisialisasi data: Program membuat struktur baru dan memasukan karakter awalsebanyak N menggunakan add_back.
- Pelacakan posisi: Program ini membuat posisi alpha dan beta kedalam bentukvariable numerik sebagai kordinatnya yaitu pos_alpha=0 (head) dan pos_beta = size- 1 (tail).
- Perulangan Ronde: menjalankan perulangan ronde sebanyak R. pada perulangan inilah proses utamanya.
- Pergerakan alpha dan beta: lompatan X dan Y dihitung dengan optimasi modulo, alphamaju dengan menambahkan sisa bagi jaraknya $\text{pos_alpha} = (\text{pos_alpha} + X) \% \text{size}$.
- Beta mundur menggunakan pengurangan yang dilindungi dengan nilai dasar sizesupaya aman dari nilai negatif $\text{pos_beta} = (\text{pos_beta} - Y \% \text{size} + \text{size}) \% \text{size}$.

- Event Tabrakan atau Resonasi:
 - jika $\text{pos_alpha} == \text{pos_beta}$ (Tabrakan): Program akan memanggil `delete_idx(pos_alpha)`. Jika list masih belum kosong posisi alpha dan beta akan otomatis terkalkulasi ulang terhadap ukuran list yang baru saja berkurang.
 - Jika tidak bertabrakan program akan cek resonansi $((\text{pos_alpha} + 1) == \text{pos_beta})$: Program akan menukar karakter dua node menggunakan `set()`.
- Output Tampilan: Setelah semua R ronde selesai list karakter langsung dicetak dari head hingga tail dengan fungsi `display()`.

Kenapa Ini Bekerja?

Algoritma ini aman karena adanya pemisahan arsitektur. Program tidak perlu melakukan pengaturan jalinan pointer list saat terjadinya tabrakan, program hanya perlu memanggil `delete_idx` yang sudah disiapkan. Fungsi yang sudah kita buat di soal sebelumnya ini akan mengubah ikatan `prev` dan `next` secara otomatis. Dengan cara ini program cukup menkalibrasi ulang `pos_alpha` dan `beta` terhadap `size` baru saja.

Mekanisme kalibrasi setelah tabrakan dan `delete_idx`, node akan bergeser satu langkah ke kiri. Karena itu posisi yang sebelumnya punya node yang dihapus sekarang sudah ditempati oleh node kanan nya. `pos_alpha` tidak perlu di ubah manual, cukup cek dengan `pos_alpha % size` supaya tidak melebihi batas.

Algoritma ini juga menangani kondisi dimensi kosong pada loop utama nya sesuai dengan instruksi dari soal, terdapat pengecekan `if (timbang → size == 0) break;` ini adalah hal yang krusial jika node sudah kosong dan program tetap berjalan ke perhitungan `X % timbang.size`, maka akan terjadi pembagian nol, yang akan membuat crash.

Analisis Kompleksitas

Kompleksitas Waktu: $O(r \cdot n)$. Karena ada loop utama (ronde) sebanyak R , dan didalamnya terdapat beberapa fungsi yang memiliki kompleksitas waktu $O(n)$, dengan demikian disetiap ronde program menghabiskan rata-rata waktu $O(n)$. Maka total nya $O(r \cdot n)$. kenapa disini tidak $O(n^2)$? Karena R disini merupakan parameter yang berbeda dari N , ya jika nilai mereka sama akan tetap $O(n^2)$ tetapi dalam kasus ini R nya beda. Jadi ini hampir sama dengan $O(n^2)$ tetapi tidak konsisten.

Kompleksitas Ruang: $O(n)$ karena algoritma menyimpan N karakter awal kedalam struktur data linked list.

MODUL 4: SORTING

SOAL 1

Sorting

Tujuan Praktikum

1. Mahasiswa mampu mengimplementasikan berbagai algoritma sorting (Comparison & Non-comparison).
2. Mahasiswa mampu melakukan analisis performa menggunakan waktu eksekusi (ms), jumlah perbandingan (comparisons), dan jumlah pertukaran (swaps).
3. Mahasiswa memahami pengaruh distribusi data terhadap efisiensi algoritma.

Ketentuan Tugas

- Implementasi dikerjakan secara modular menggunakan template yang disediakan.
- Dilarang keras menggunakan library `std::sort`. Semua algoritma harus ditulis manual.
- Dilarang menggunakan AI. Ini kalian harus analisis sendiri. Baca dulu dari sumber-sumber buku, materi, internet yang diberikan
- Sortingnya dengan urutan menaik atau ascending
- Data yang akan diurutkan berupa `vector<int>`

Algoritma Sorting

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Radix Sort

Instruksi Pengujian

Lakukan pengujian terhadap setiap algoritma menggunakan beberapa kondisi data berikut:

1. Data acak (random).
2. Data sudah (atau hampir) terurut menaik.
3. Data terurut menurun.

4. Data dengan banyak elemen duplikat.

Hal yang Perlu Dianalisis

Untuk setiap algoritma, lakukan analisis terhadap aspek berikut:

1. Kompleksitas waktu:
 - Best Case
 - Average Case
 - Worst Case
2. Karakteristik algoritma:
 - Apakah algoritma bersifat stable atau tidak.
 - Apakah algoritma termasuk in-place atau membutuhkan memori tambahan.
 - Mekanisme utama algoritma dalam melakukan pengurutan.
3. Analisis operasi:
 - Jumlah comparison.
 - Jumlah swap.
 - Jumlah shift (khusus algoritma tertentu seperti Insertion Sort).
 - Jumlah recursion call.
 - Jumlah array accessed.
 - Pengaruh bentuk data terhadap jumlah operasi.
4. Analisis performa:
 - Perbandingan waktu eksekusi antar algoritma.
 - Algoritma mana yang paling cepat untuk dataset kecil dan besar.
 - Pengaruh distribusi data terhadap performa algoritma.
 - Hubungan antara teori kompleksitas dan hasil eksperimen yang diperoleh.

A. Source Code

sorting_algorithms.cpp

1	#include "sorting_algorithms.h"
2	#include <algorithm>
3	#include <chrono>

```

4
5 using Clock = std::chrono::high_resolution_clock;
6
7 void bubble_sort(std::vector<int> &data, Metrics &m) {
8     int n = data.size();
9     auto start = Clock::now();
10
11     // Done: implementasi Bubble Sort
12     for (int i = 0; i < n - 1; i++) {
13         // Loop kedua untuk membandingkan elemen
14         for (int j = 0; j < n - i - 1; j++) {
15             m.comparisons++;
16             // Jika elemen lebih besar dari elemen berikutnya,
17             tukar
18             if (data[j] > data[j + 1]) {
19                 std::swap(data[j], data[j + 1]);
20                 m.swaps++;
21             }
22         }
23     }
24
25     m.time_ms =
26         std::chrono::duration<double,
27         std::milli>(Clock::now() - start).count();
28 }
29
30 void selection_sort(std::vector<int> &data, Metrics &m)
31 {
32     int n = data.size();
33     auto start = Clock::now();
34
35     // Done: implementasi Selection Sort
36     for (int i = 0; i < n - 1; i++) {
37         // Asumsi elemen pertama adalah elemen terkecil
38         int min_idx = i;
39         // Loop kedua untuk mencari elemen terkecil di
40         array
41         for (int j = i + 1; j < n; j++) {
42             m.comparisons++;
43             // Jika elemen lebih kecil dari elemen terkecil,
44             update elemen terkecil
45             if (data[j] < data[min_idx]) {
46                 min_idx = j;
47             }
48         }
49         // Tukar elemen terkecil dengan elemen pertama
50         std::swap(data[i], data[min_idx]);
51         m.swaps++;

```

```

52     }
53     // Gunakan m.comparisons++ dan m.swaps++
54
55     m.time_ms =
56         std::chrono::duration<double,
57         std::milli>(Clock::now() - start).count();
58 }
59
60 void insertion_sort(std::vector<int> &data, Metrics &m)
61 {
62     int n = data.size();
63     auto start = Clock::now();
64
65     // Done: implementasi Insertion Sort
66     for (int i = 1; i < n; i++) {
67         int key = data[i];
68         int j = i - 1;
69
70         while (j >= 0 && data[j] > key) {
71             m.comparisons++;
72             // Geser elemen ke kanan
73             data[j + 1] = data[j];
74             m.shifts++;
75             j--;
76         }
77         // Masukkan key ke posisi yang tepat
78         data[j + 1] = key;
79         m.shifts++;
80     }
81     // Gunakan m.comparisons++ dan m.shifts++
82
83     m.time_ms =
84         std::chrono::duration<double,
85         std::milli>(Clock::now() - start).count();
86 }
87
88 void merge(std::vector<int> &data, int left, int mid, int
89 right, Metrics &m) {
90     // Mencari ukuran sub array left dan right
91     int n1 = mid - left + 1;
92     int n2 = right - mid;
93
94     // Membuat sub array
95     std::vector<int> L(n1), R(n2);
96     // Memasukkan data ke sub array
97     for (int i = 0; i < n1; i++)
98         L[i] = data[left + i];
99     for (int j = 0; j < n2; j++)

```

```

100     R[j] = data[mid + 1 + j];
101     // Variable helper untuk track index sub array dan main
102     array
103     int i = 0, j = 0, k = left;
104     // Membandingkan elemen sub array L dan R, lalu
105     memasukkan ke main array
106     while (i < n1 && j < n2) {
107         m.comparisons++;
108         // Jika elemen sub array L lebih kecil dari elemen
109         sub array R
110         // Maka masukkan elemen sub array L ke main array
111         // Jika tidak, masukkan elemen sub array R ke main
112         array
113         if (L[i] <= R[j]) {
114             // Memasukkan elemen sub array L ke main array
115             data[k] = L[i];
116             i++;
117         } else {
118             // Memasukkan elemen sub array R ke main array
119             data[k] = R[j];
120             j++;
121         }
122         k++;
123     }
124     // Memasukkan elemen sub array L yang tersisa
125     while (i < n1) {
126         data[k] = L[i];
127         i++;
128         k++;
129     }
130     // Memasukkan elemen sub array R yang tersisa
131     while (j < n2) {
132         data[k] = R[j];
133         j++;
134         k++;
135     }
136 }
137
138 void merge_sort_helper(std::vector<int> &data, int left,
139 int right,
140                     Metrics &m) {
141     m.recursive_calls++;
142
143     // Basis kasus: Jika sub array kosong atau memiliki 1
144     elemen, maka sudah
145     // terurut
146     if (data.empty())
147         return;

```

```

148     if (left >= right)
149         return;
150     // Membagi array menjadi 2 bagian
151     int mid = left + (right - left) / 2;
152     // Memanggil recursive function untuk sub array kiri
153     merge_sort_helper(data, left, mid, m);
154     // Memanggil recursive function untuk sub array kanan
155     merge_sort_helper(data, mid + 1, right, m);
156     // Menggabungkan sub array yang sudah terurut
157     merge(data, left, mid, right, m);
158 }
159
160 void merge_sort(std::vector<int> &data, Metrics &m) {
161     auto start = Clock::now();
162
163     // Done: panggil helper rekursif di sini
164     merge_sort_helper(data, 0, data.size() - 1, m);
165     // Gunakan m.comparisons++ dan m.recursive_calls++
166
167     m.time_ms =
168         std::chrono::duration<double,
169         std::milli>(Clock::now() - start).count();
170 }
171
172 int quick(std::vector<int> &data, int low, int high,
173 Metrics &m) {
174     int pivot = data[high];
175     int i = low - 1;
176
177     // loop i = -1 j = 0
178     // data = [5, 2, 8, 1, 9, 3, 7]
179     // pivot = 7
180     // it 1 = no swap
181     // it 2 = no swap i = 1 j = 1
182     // it 3 = j = 2 i = 1 Skip
183     // it 4 = j= 3 i = 2 swap data[2], data[3] = [5, 2, 1,
184 8, 9, 3, 7]
185     // it 5 = j = 4 i = 2 skip
186     // it 6 = j = 5 i = 3 swap data[3], data[5] = [5, 2, 1,
187 3, 9, 8, 7]
188     for (int j = low; j < high; j++) {
189         m.comparisons++;
190         if (data[j] <= pivot) {
191             i++;
192             if (i != j) {
193                 std::swap(data[i], data[j]);
194                 m.swaps++;
195             }

```

```

196     }
197 }
198 // swap data[4] dan data[6] = [5, 2, 1, 3, 7, 8, 9]
199 std::swap(data[i + 1], data[high]);
200 m.swaps++;
201 // pi 4
202 return i + 1;
203 }
204
205 void quick_sort_helper(std::vector<int> &data, int low,
206 int high, Metrics &m) {
207     m.recursive_calls++;
208     if (low < high) {
209         // pi 4
210         int pi = quick(data, low, high, m);
211         // recursive calls di sini
212         // pivot baru jadi high nya left dan low nya right
213         // left = [5, 2, 1, 3] pi-1 = 3
214         // it 1 = no swap j = 0 i = -1
215         // it 2 = j = 1 i = 0 swap data[0], data[1] = [2, 5,
216 1, 3]
217         // it 3 = j = 2 i = 1 swap data[1], data[2] = [2, 1,
218 5, 3]
219         // swap data[2], data[3] = [2, 1, 3, 5]
220         // pi 2
221         // Recursive calls di sini lagi
222         quick_sort_helper(data, low, pi - 1, m);
223         // right = [8, 9, 7] high = 6 pi+1 = 5
224         quick_sort_helper(data, pi + 1, high, m);
225     }
226 }
227
228 void quick_sort(std::vector<int> &data, Metrics &m) {
229     auto start = Clock::now();
230
231     // Done: panggil helper rekursif di sini
232     quick_sort_helper(data, 0, data.size() - 1, m);
233     // Gunakan m.comparisons++, m.swaps++, dan
234 m.recursive_calls++
235
236     m.time_ms =
237         std::chrono::duration<double,
238 std::milli>(Clock::now() - start).count();
239 }
240
241 // TODO: tambahkan helper per-digit sort di sini jika
242 perlu
243 void counting_sort_by_digit(std::vector<int> &data, int

```

```

244 exp, Metrics &m) {
245     int n = data.size();
246     std::vector<int> output(n);
247     int count[10] = {0};
248     // Awal data = [170, 45, 75, 90, 2, 802, 24, 66]
249     //      Index   0    1    2    3    4    5    6    7
250
251     // exp = 10
252     //   [170, 90, 2, 802, 24, 45, 75, 66]
253     // Hitung frekuensi digit
254     for (int i = 0; i < n; i++) {
255         int digit = (data[i] / exp) % 10;
256         count[digit]++;
257         m.array_accesses++;
258     }
259     //   count = [2, 0, 1, 0, 1, 0, 1, 2, 0, 1, 0]
260     //           [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
261
262     // Prefix sum untuk posisi
263     for (int i = 1; i < 10; i++) {
264         count[i] += count[i - 1];
265         m.array_accesses++;
266     }
267     //   count = [2, 2, 3, 3, 4, 4, 5, 7, 7, 8, 8]
268     //   index = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
269
270     // Build output vector (traverse dari belakang agar
271     stabil)
272     for (int i = n - 1; i >= 0; i--) {
273         int digit = (data[i] / exp) % 10;
274         output[count[digit] - 1] = data[i];
275         count[digit]--;
276         m.array_accesses++;
277     }
278     //   count = [0, 2, 2, 3, 3, 4, 4, 5, 7, 7, 8]
279     //   index = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
280
281     // output = [2, 802, 24, 45, 66, 170, 75, 90]
282     //   Index   0    1    2    3    4    5    6    7
283
284     // Copy kembali ke data
285     for (int i = 0; i < n; i++) {
286         data[i] = output[i];
287         m.array_accesses++;
288     }
289 }
290
291 // exp 100

```

```

292 // output = [2, 802, 24, 45, 66, 170, 75, 90]
293 // freq digi : for (int i = 0; i < n); int digit =
294 (data[i] / exp) % 10;
295 // count[digit]++;
296 // count = [6, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0]
297 // index [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
298 // pref sum count : for (int i = 1; i < 10; i++) count[i]
299 += count[i - 1];
300 // count = [6, 7, 7, 7, 7, 7, 7, 7, 8, 8, 8]
301 // index [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
302 // Build output : for (int i = n - 1; i >= 0; i--)
303 // int digit = (data[i] / exp) % 10; output[count[digit]
304 - 1] = data[i];
305 // count[digit]--;
306 // count = [0, 6, 7, 7, 7, 7, 7, 7, 7, 8, 8]
307 // index [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
308 // output = [2, 24, 45, 66, 75, 90, 170, 802]
309 // Index [0, 1, 2, 3, 4, 5, 6, 7]
310 // Sorted
311 void radix_sort(std::vector<int> &data, Metrics &m) {
312     if (data.empty())
313         return;
314     auto start = Clock::now();
315
316     // Done: implementasi Radix Sort
317     // mencari max element
318     int max_val = *std::max_element(data.begin(),
319 data.end());
320     for (int exp = 1; max_val / exp > 0; exp *= 10) {
321         counting_sort_by_digit(data, exp, m);
322     }
323     // Gunakan m.array_accesses++
324
325     m.time_ms =
326         std::chrono::duration<double,
327 std::milli>(Clock::now() - start).count();
328 }

```

Table 19. Source Code sorting_algorithms.cpp

B. Pembahasan

Bubble Sort

Kompleksitas waktu

1. **Best Case** : Kasus terbaik untuk bubble sort adalah pada bentuk data yang sudah urut dengan jumlah swap 0 dan waktu 0.003ms.
2. **Average Case** : Kasus rata – rata dari algoritma ini berada pada bentuk data yang banyak duplikat. Dengan jumlah swap 2315 dan waktu 0.02ms.
3. **Worst Case** : Kasus terburuk dari algoritma ini adalah pada bentuk data reverse sorted dengan jumlah swap 4950 dan waktu 0.029ms

Untuk Big-O notation nya adalah $O(n^2)$. Karena terdapat nested loop sebanyak n pada algoritmanya pada baris [12,14], dan tidak ada perubahan pada jumlah comparisons dari semua jenis bentuk data yaitu 4950. membuktikan bahwa tidak ada perbedaan notasi walaupun bentuk data berubah.

Karakteristik algoritma

Apakah algoritma bersifat stable karena element yang sama posisi nya tidak akan dilakukan pertukaran, dan posisi nya tetap relatif pada sorting, hal ini karena pada comparasi pada baris [18] algoritma menggunakan $>$ sehingga jika nilai sama maka tidak akan di swap. Algoritma ini inplace karena dia tidak memerlukan array / vector / container bantuan dengan besar sebanyak n , dan langsung melakukan sorting pada vector / data aslinya.

Algoritma ini bekerja dengan membandingkan semua data satu persatu data yang ada dengan data disebelahnya, jika nilai pada index saat ini lebih kecil dari data setelah nya swap mereka terus menerus sampai algoritma selesai membandingkan semua nilai setiap index pada data

Analisis operasi

Comparison : 4950 untuk semua bentuk data

Swap : Random sebanyak 2564, Sorted sebanyak 0, Reverse Sorted sebanyak 4950, Nearly Sorted sebanyak 296, Many Duplicates sebanyak 2315.

Pengaruh bentuk data terhadap jumlah operasi :

Data terurut menaik : ini menjadi best case karena algoritma tidak perlu lagi melakukan swap karena data sudah terurut dan ini disebabkan oleh comparasi yang ada pada baris [18].

Data terurut menurun : menjadi worst case karena data akan di swap sama banyaknya dengan comparison karena hasil dari comparison nya akan selalu true sehingga swap selalu dilakukan.

Data acak : terlihat hasil data acak memiliki swap yang di atas rata – rata hal ini karena jika data sama maka swap tidak akan dilakukan hal ini membuat swap secara natural lebih sedikit dari swap data banyak duplikat.

Data banyak duplikat : ini menjadi kasus rata – rata karena ada beberapa swap yang tidak dilakukan sebab komparasi yang memiliki hasil false sehingga ada beberapa ini menjadi lebih dekat dari random.

Selection Sort

Kompleksitas waktu

1. **Best Case** : Sorted menjadi best case dengan waktu 0.003296ms.
2. **Average Case** : Reverse Sorted average case dengan waktu 0.004057ms.
3. **Worst Case** : Random menjadi worst case dengan waktu 0.006433ms.

Untuk algoritma ini sebenarnya perbedaan hanya terdapat pada waktu komparasi saja, Big-O notation dari algoritma ini adalah $O(n^2)$ karena adanya nested loop sebanyak n pada baris [36, 41], dan tidak ada perubahan notasi walaupun bentuk data berbeda karena komparasi dan swap memiliki nilai yang sama pada semua bentuk data.

Karakteristik algoritma

Algoritma ini bersifat tidak stable karena dia akan menukar index data jika nilai nya sama jadi posisi relatif dari nilai yang sama tidak bisa ditentukan. Algoritma termasuk in-place karena tidak memerlukan array / vector (container n) bantuan dia hanya memerlukan 1 variabel bantuan pada baris [33].

Pertama algoritma ini akan mengasumsi bahwa index pertama adalah nilai paling kecil. Kemudian akan dilakukan perulangan i untuk mengisi setiap index dari data, yang didalamnya ada perulangan j lagi untuk mencari nilai terkecil. Pada perulangan mencari nilai terkecil ini terdapat komparasi dimana jika nilai index saat ini lebih kecil maka nilai terkecil di update untuk menyimpan nilai index yang baru. Dan jika sudah semua index di cek swap nilai index terkecil dengan index i (index perulangan i saat ini). Dan ini dilakukan terus sampai semua index pada perulangan i sudah selesai.

Analisis operasi

Comparison : 4950 untuk semua bentuk data

Swap : 99 untuk semua bentuk data.

Pengaruh bentuk data terhadap jumlah operasi pada algoritma ini tidak terlalu berdampak karena bagaimana pun bentuk data nya algoritma ini akan selalu melakukan swap yang sama yaitu sebanyak $n-1$, yang menjadi pembeda antara bentuk data disini hanya pada waktunya, dimana Random 0.006ms, Sorted 0.003ms, Reverse Sorted 0.004ms, Nearly Sorted 0.003ms,

Many Duplicates 0.004ms. Hal ini terjadi karena waktu comparison, jika makin random maka makin lama waktu comparison nya.

Insertion Sort

Kompleksitas waktu

1. **Best Case** : $O(n)$. Sorted menjadi best case karena comparisons nya 0 dan shifts nya 99.
2. **Average Case** : $O(n^2)$. Random menjadi average case karena comparisons nya 2564 dan shifts nya 2663.
3. **Worst Case** : $O(n^2)$. Reverse Sorted menjadi worst case karena comparisons nya 4950 dan shifts nya 5049.

Kenapa case nya ini sangat berbeda pada bentuk data nya?, ini karena algoritma ini menggunakan perulangan berkondisi yaitu while pada baris [70-80]. ini membuat jika key lebih besar dari data j maka shifts tidak perlu dilakukan, dan semakin kanan kamu perlu melakukan while shifts ini makan makin banyak comparison dan shifts nya. Karena ini Big-O Notation nya berbeda pada Best case $O(n)$, karena kita tidak perlu melakukan loop while, dan pada average dan worst $O(n^2)$ karena loop while dijalankan.

Karakteristik algoritma

Algoritma ini bersifat stable karena comparison nya menggunakan $>$ dimana ini membuat algoritma tidak akan melakukan shift jika data dan key nya sama dia tidak akan melakukan shift. Algoritma ini termasuk in-place karena algoritma ini tidak membuat container pembantu sebesar n dan sorting dilakukan pada container data asli..

Pertama algoritma ini akan melakukan perulangan sebanyak n , yang membuat perulangan ini menarik adalah dia mulai dari 1 tidak 0, karena key mulai dari index 1. perulangan ini digunakan sebagai bagian utama algoritma. Didalam perulangan itu algoritma membuat variable bantu bernama key dan j. Key digunakan untuk menyimpan data pada index 1, dan j digunakan untuk menyimpan index dari $i - 1$ dari perulangan pertama. Kemudian ada perulangan while dengan kondisi $j >= 0$ dan data index $j > \text{key}$. Selama kondisi benar maka kita majukan data index nya dengan membuat data index $j+1 = \text{data } j$ dan mengurangi nilai $j - 1$ dengan begitu index belakang j akan maju 1 index selama kondisi benar. Jika kondisi sudah tidak terpenuhi maka lanjut untuk masukan data index $j + 1$ setelah j menjadi -1 jadi kita kembalikan menjadi 0 dan kita isi dengan nilai key.

Analisis operasi

Comparison : Random sebanyak 2564, Sorted sebanyak 0, Reverse Sorted sebanyak 4950, Nearly Sorted sebanyak 296, Many Duplicates sebanyak 2315.

Shifts : Random sebanyak 2663, Sorted sebanyak 99, Reverse Sorted sebanyak 5049, Nearly Sorted sebanyak 395, Many Duplicates sebanyak 2414.

Pengaruh bentuk data terhadap jumlah operasi:

Data terurut menaik : Sorted memiliki comparison dan shifts paling rendah karena while loop langsung false. Loop dan shifts nya nggak jalan.

Data terurut menurun : Reverse Sorted menjadi yang worst karena key harus geser lewati semua elemen di kiri.

Data acak : Random ini menjadi average karena key kadang tidak perlu geser sepenuhnya hanya sampe tempatnya.

Data banyak duplikat : Banyak duplikat ini ditengah – tengah karena kalau nilai sama, kondisi $> \text{false}$. Jadi elemen nggak perlu geser melewati elemen yang nilainya sama.

Kalau datanya udah rapi (sorted), Insertion Sort ini cepat. Comparisons sama shifts-nya cuma sekitar 99. Tapi kalau datanya terbalik, dia langsung lemot. Tiap angka baru harus diselipin ke paling depan, jadi key dan datanya shifts terus. Untuk data acak, dia di tengah-tengah aja. Yang menarik, kalau data banyak angka yang sama (duplikat), dia sedikit lebih cepat karena tidak perlu tuker posisi kalau nilainya sama.

Merge Sort

Kompleksitas waktu

1. **Best Case, Average Case, Worst Case** : Sama karena Recursive Calls nya sama pada semua bentuk data.

Karena bagaimana pun datanya, algoritma ini tetep bagi 2, bagi 2, bagi 2... terus merge. Nggak ada jalan pintas. Jadi jumlah operasi nya relatif konstan. Tapi kalau dilihat dari jumlah comparisons pas merge, ada perbedaan kecil. Meski begitu, perbedaannya cuma ratusan, nggak mengubah notasi Big-O yang tetep $O(N \log N)$.

Karakteristik algoritma

Algoritma bersifat stable karena merge, adanya comparasi if ($L[i] \leq R[j]$). Tanda $\leq$ ini karena jika elemen kiri dan kanan nilainya sama, yang dari kiri (L) yang diambil duluan. Jadi posisi relatif elemen bernilai sama nggak berubah di datanya. Algoritma ini tidak in-place karena algoritma ini perlu vector bantuan (L dan R). dan juga recursive calls nya sebanyak n juga. Space complexity-nya $O(N)$ karena ini tidak nested recursive calls dan vector nya. karena bergantung sebanyak n walaupun dibagi-bagi vectornya

Algoritma ini bekerja dengan cara memanggil fungsi recursive yang pada kasus ini adalah helper yang memiliki parameter data, left, right. Dimana pemanggilan pertama left nya adalah 0 dan right nya data size -1. ini adalah panggilan awal dimana data belum dibagi. Panggilan ini kita ibaratkan menjadi stack panggilan fungsi. Sebelum itu fungsi helper disini memiliki cek apakah data empty dan cek $\text{left} \geq \text{right}$ dimana size nya disini 1 dia akan return dan mematikan jalan fungsi, dan fungsi ini juga membuat var mid yang berfungsi sebagai pembagi data ditengah nya. Kemudian fungsi ini juga memanggil dirinya sendiri 2x dengan parameter yang berbeda dimana yang kiri `merge_sort_helper(data, left, mid, m)`; right nya menjadi mid dan yang kanan `merge_sort_helper(data, mid + 1, right, m)`; dimana left nya menjadi mid + 1. artinya fungsi ini akan terus membagi vector data sampai vector datanya hanya 1. kemudian fungsi ini juga memanggil merge yang akan melakukan merge vector data.

Jadi alur nya adalah helper utama masuk ke stack, dia akan memanggil helper 1 kiri dlu dan masuk stack, dan helper 2 kiri dlu, nah jika helper 2 kiri ini size nya 1 dia akan return dan menutup stack ini, kemudian pada helper 1 kiri akan memanggil helper 2 kanan, dan jika size 1 juga kembali, baru dia akan merge dengan parameter helper 1 kiri dan seterusnya sampai merge pada helper utama.

Untuk merge? Algoritma merge biasa.

Analisis operasi

Comparison : Random sebanyak 535, Sorted sebanyak 356, Reverse Sorted sebanyak 316, Nearly Sorted sebanyak 453, Many Duplicates sebanyak 536.

Rec.Calls : Random sebanyak 199, Sorted sebanyak 199, Reverse Sorted sebanyak 199, Nearly Sorted sebanyak 199, Many Duplicates sebanyak 199.

Pengaruh bentuk data terhadap jumlah operasi.

Untuk Merge Sort, bentuk data tidak terlalu berpengaruh ke jumlah operasi secara signifikan. Recursive calls-nya tetep 199 dengan $N=100$. Namun comparison nya yang sedikit berbeda hal ini terjadi karena jika datanya udah sorted atau reverse, saat merge salah satu subvector cepat habis, jadi comparisons nya lebih dikit. Jika datanya acak dan elemennya saling berselipan, comparisons nya jadi maksimal karena loop while jalan terus.

Quick Sort

Kompleksitas waktu

1. **Best Case** : $O(N \log N)$. Random dan Many duplicate menjadi base case, karena pivot selalu membagi vector jadi 2 bagian yang seimbang (sekitar $n/2$).

2. **Average Case** : $O(N \log N)$. Random menjadi case yang average karena partisi-nya cenderung seimbang.
3. **Worst Case** : $O(N^2)$. Sorted dan Reverse Sorted menjadi case paling buruk karena pivot selalu jadi elemen terbesar/terkecil. Partisi nggak seimbang, dan tidak bisa di partisi, jadi recursive depth-nya jadi N .

Karakteristik algoritma

Algoritma ini bersifat tidak stable. Karena saat partition quick, elemen bisa loncat posisi lewat nilai i dan j , jadi urutan relatif elemen yang nilainya sama bisa berubah.

Algoritma ini termasuk in-place (sorting dilakukan di vector asli), tapi butuh stack memori untuk recursive calls .

Pertama ada fungsi quick disini sebagai fungsi sorting nya, pertama fungsi ini memiliki parameter data, low, dan high. Pada fungsi ini kita perlu var pivot dengan value = high dalam contoh ini 6, dan $I = \text{low} - 1$ ($\text{low} = 0$) / $I = -1$. kemudian ada perulangan utama dari fungsi ini dengan kondisi $\text{int } j = \text{low}; j < \text{high}; j++$, didalam nya ada comparison $\text{data}[j] \leq \text{pivot}$ dimana jika true nilai I akan di increment dan akan dilakukan $\text{swap}(\text{data}[i], \text{data}[j])$, increment I hanya akan jalan apabila $j \leq \text{pivot}$ ini menyebabkan I tertinggal oleh j yang tetap terus di increment ini membuat swap tidak swap dengan dirinya sendiri. Masih di fungsi ini setelah perulangan selesai akan ada swap ($\text{data}[i + 1], \text{data}[\text{high}]$), I disini nilai baru setelah increment dari loop, dan kemudian fungsi ini akan return $i + 1$ yang nantinya digunakan sebagai pivot baru.

Sekarang ke fungsi helper tempat dimana ada nya recursive calls untuk membagi panjang vector sehingga fungsi quick disini tidak perlu melakukan banyak iterasi di loop nya karena panjang vector yang di pangkas dengan mid nya sesuai dengan pivot. Pertama didalam fungsi ini ada cek jika $\text{low} < \text{high}$, dan didalam nya ada var pi yang menampung hasil dari quick yang saya jelaskan diatas, dan ada 2 recursive calls yaitu left dan right, yang membedakan dari merge disini mid nya akan di isi oleh pi yang nanti nya jadi pivot baru di quick.

Analisis operasi

Comparison: Random sebanyak 676, Sorted sebanyak 4950, Reverse Sorted sebanyak 4950, Nearly Sorted sebanyak 2638, Many Duplicates sebanyak 832, comp ini terjadi di setiap perbandingan elemen dengan pivot di dalam loop partition quick baris [190].

Swap: Random sebanyak 396, Sorted sebanyak 5049, Reverse Sorted sebanyak 2549, Nearly Sorted sebanyak 2607, Many Duplicates sebanyak 707, terjadi saat elemen dipindahkan ke posisi i baris [193] dan saat pivot dipasang di posisi akhir baris [199].

Recursion call: Random sebanyak 131, Sorted sebanyak 199, Reverse Sorted sebanyak 199, Nearly Sorted sebanyak 191, Many Duplicates sebanyak 181, terjadi setiap masuk ke `quick_sort_helper` baris [222, 224], termasuk base case.

Pengaruh bentuk data terhadap jumlah operasi.

Dari hasil matriks nya ini quick sort cenderung susah di tipe data yang sudah terurut karena pivot akan selalu mengambil right yang dimana jika datanya terurut maka biasanya pivot menjadi yang paling besar atau kecil ini membuat partisi nya tidakimbang atau bahkan hanya bisa menghasilkan 1 partisi.

Radix Sort

Kompleksitas waktu

1. **Best Case, Average Case, Worst Case:** Tidak ada perbedaan yang didasarkan tipe data, hal ini karena sort dilakukan seberapa banyak tergantung dengan digit dari max element pada baris [254, 255], dan loop didalam sort akan tetap dilakukan sebanyak n kali tanpa kondisi apapun. Dengan begitu kompleksitas waktu nya $O(n \times d)$, d adalah digit max element, dan Big-O notation nya menjadi $O(n)$ karena d adalah static tidak tergantung n .

Karakteristik algoritma

Algoritma ini bersifat stable. Karena counting sort yang digunakan membangun output dari belakang (traverse mundur), sehingga elemen yang muncul lebih dulu tetap didepan karena yang belakang dimasukan terlebih dahulu.

Algoritma ini bukan in-place. Dia butuh container bantuan vector output dan array `count[10]`, sehingga kompleksitas ruang nya $O(n + k)$ dengan $k=10$, Big-O notation nya menjadi $O(n)$

Algoritma ini sangat berbeda dari algoritma sebelumnya karena algoritma ini tidak menggunakan comparison terhadap dua data sama sekali, tetapi dia langsung mengurutkan data berdasarkan nilai satuan, puluhan, ratusan, dan seterusnya, data. Algoritma ini perlu tahu element terbesar dari data yang ingin di sort, data ini akan menjadi patokan seberapa banyak algoritma ini akan dijalankan. Algoritma ini memiliki parameter data dan `exp`, `exp` ini adalah 10 dan iterasi $\times 10$, jadi jika max value nya ratusan maka akan jalan $3x$.

Algoritma ini memerlukan 2 container bantuan, 1 container array `count` dengan size 10, dan 1 vector dengan size n , algoritma ini akan melakukan 4 loop dengan tujuan yang berbeda. Loop pertama untuk mencari frekuensi digit (digit nilai di data) yang akan dimap ke `count`, loop ini berjalan dengan kondisi $i < n$ dengan iterasi $i++$, pertama pada loop ini perlu mencari digit data dengan cara $(data[i] / exp) \% 10$, kemudian nilai `count[digit]` di tambah 1. Loop kedua untuk menjumlahkan nilai yang ada pada `count`, loop ini berjalan dengan $i < 10$; $i++$, didalam nya kita akan memasukan nilai `count[i]` menjadi `count[i]+count[i - 1]`. Loop

ketiga untuk melakukan assign data ke vector output, loop ini berjalan dengan $i = n - 1; i \geq 0; i--$, ya loop ini mulai dari belakang agar dia lebih stabil karena jika ada element yang sama posisi yang didepan akan relatif tetap didepan antara element yang sama, loop ini tetap memerlukan digit dari data $(data[i] / exp) \% 10$, kemudian lakukan assign $output[count[digit] - 1] = data[i]$ artinya index output tergantung nilai-1 yang ada di count dengan index digit nya, kemudian nilai yang ada pada $count[digit]--$ dikurangi satu. Loop keempat adalah loop standar yang assign nilai output satu per satu ke data.

Analisis operasi

Array Access: Random sebanyak 927, Sorted sebanyak 927, Reverse Sorted sebanyak 927, Nearly Sorted sebanyak 927, Many Duplicates sebanyak 618, array acces dilakukan setiap kali assign value ke vector bantuan count dan juga saat memasukan value sorted ke vector bantuan output, begitu juga dengan data. Dengan demikian array acces menjadi metode operasi utama dari algoritma ini

Pengaruh bentuk data terhadap jumlah operasi.

Bentuk data nggak terlalu ngaruh. Jumlah pass ditentukan oleh digit max, bukan urutan data. Data yang udah sorted, acak, atau reverse tetep jalan 3 pass kalau max-nya 802. Yang beda mungkin distribusi digit tertentu, tapi Big-O tetap $O(N \times D)$ $D = \text{digit max element}$.

Analisis performa

Algoritma	Waktu Eks dengan N = 100 (Random)	Peringkat
Bubble Sort	0.023644 ms	6
Selection Sort	0.006803 ms	5
Insertion Sort	0.002064 ms	1
Merge Sort	0.008476 ms	4
Quick Sort	0.003567 ms	2
Radix Sort	0.006292 ms	3

Table 20. Table waktu eksekusi Modul 4

"Berdasarkan tabel di atas, algoritma yang paling cepat untuk dataset random dengan $N=100$ adalah Insertion Sort, dengan waktu 0.002064 ms. Ini terjadi karena insertion sort memiliki mekanisme exit early dimana jika data yang mau di insert sudah pada posisi yang benar dia akan berhenti melakukan shift. Di sisi lain, Bubble Sort paling lambat dengan waktu 0.023644 ms. Ini karena algoritma ini memang memiliki notasi $O(n^2)$ dimana dia akan melakukan iterasi walaupun satu per satu untuk membandingkan data.

Ukuran Data	Algoritma tercepat	Alasan
Kecil N = 100	Insertion Sort 0.002064 ms	Karena insertition hanya perlu melakukan sedikit shift jika ukuran data nya kecil, dan dia juga punya mekanisme early exit pada loop shift nya.
Besar N = 10000	Radix Sort 0.482582 ms	Karena algoritma ini memiliki kompleksitas $O(n)$ dan karena hanya memiliki 1 jenis operasi yang simple.

Table 21. Table algoritma tercepat Modul 4

Untuk data kecil insertion menjadi yang tercepat karena dia tidak perlu melakukan loop shift yang banyak hanya untuk mencari posisi satu data. Karena loop shift ini bekerja dengan shift satu per satu dari belakang hingga posisi yang benar ini membuat data yang dibelakang waktu loop nya jadi lama. Hal ini membuat cara loop dan shift ini menjadi kelemahan jika datanya banyak. Sedangkan Radix memiliki karakteristik yang stabil dimana waktu nya hanya dipengaruhi oleh n , dan tidak ada kesenjangan operasi antara n besar dan n kecil k . karena radix melakukan operasi array acces dan tidak melakukan comparasi seperti algoritma lain.

Bentuk Data	Algoritma	Alasan
Sangat Sensitif	Insertion Sort dan Quick Sort	Dua algoritma ini berlawanan dimana insertion semakin urut datanya semakin cepat, bebanding dengan quick sort jika data nya urut ata urut terbalik dia menjadi lambat
Tidak Sensitif	Merge Sort	Karena algoritma ini menggunakan divide membuat nya secara natural tidak terpengaruh pada bentuk data

Table 22. Table algoritma sensitif bentuk data Modul 4

Tipe Data N = 10.000	Insertion Sort	Quick Sort	Merge Sort
Sorted	0.007464 ms	40.883075 ms	0.432948 ms
Reverse Sorted	87.947053 ms	46.149733 ms	0.576839 ms

Random	19.505480 ms	0.487881 ms	0.927542 ms
Nearly Sorted	5.566374 ms	0.694992 ms	0.768351 ms
Many Duplicates	44.108827 ms	0.463816 ms	1.043040 ms

Table 23. Table algoritma sensitif bentuk data waktu Modul 4

Karena insertion ini melakukan loop shift yang memiliki karakteristik semakin kekanan data nya semakin banyak shift yang perlu dilakukan, karakteristik ini membuat algoritma ini sangat tidak efisien untuk tipe data reverse sorted yang besar, sebaliknya jika data nya sorted algoritma ini sangat cepat. Yang menarik disini adalah quick sort dimana semakin data nya sorted semakin tidak efisien. Sedangkan merge sorted sangat stabil tidak peduli tipe data ataupun besar data nya. Hal ini karena dia melakukan divide and conquer, dimana data akan terus di divide menjadi 2 sampai data nya sisa 1 dan kemudian dia baru melakukan merge.

Algoritma	Big-O	N=100	N=1.000	N=10.000	Sesuai teori?
Bubble Sort	$O(n^2)$	0.023644	1.715169	216.477113	Ya dengan lonjakan 72 dan 9100
Selection Sort	$O(n^2)$	0.006803	0.287483	25.555336	Lebih Sedikit dengan lonjakan 42 dan 3700
Insertion Sort	$O(n^2)$	0.002064	0.133402	19.505480	Ya dengan lonjakan 64 dan 9400
Merge Sort	$O(n \log n)$	0.008476	0.078769	0.927542	Lebih Sedikit dengan lonjakan 9 dan 109
Quick Sort	$O(n \log n)$	0.003567	0.044243	0.487881	Ya dengan lonjakan 12 dan 136
Radix Sort	$O(n)$	0.006292	0.041498	0.482582	Lebih Sedikit dengan lonjakan 6 dan 76

Table 24. Table hubungan teori dan hasil Modul 4

Validasi: Apakah waktu eksekusi naik sesuai nilai Big-O?

Untuk $O(n^2)$: kalau N 10 kali lipat (100 → 1.000 → 10.000), waktunya seharusnya naik sekitar 100 kali lipat (10^2).

Untuk $O(n \log n)$: kalau N 10 kali lipat, waktunya naik sekitar $10 \times (\log 10N / \log N)$ kali, kurang lebih 13-15 kali.

Untuk $O(n)$: waktunya naik linear (10 kali lipat kalau N 10 kali lipat).

Untuk beberapa algoritma, seperti bubble, insertion, dan quick hasil big-o cukup akurat karena rasio kenaikan waktu pada skala besar ($N = 10.000$) sudah mendekati prediksi teori, di mana Bubble dan Insertion menunjukkan lonjakan sekitar 9.000–9.500 kali (mendekati teori $N^2 = 10.000$ kali) serta Quick Sort mengikuti pola $N \log N$; sedangkan Selection Sort memiliki rasio yang lebih rendah karena jumlah swap-nya selalu tetap sebesar $N-1$ sehingga beban operasi pertukaran mendominasi sebagian waktu eksekusi di semua skenario, Merge Sort hasilnya lebih lambat dalam rasio karena setiap pemanggilan merge harus membuat dan menyalin subvector tambahan yang membebani eksekusi di setiap level rekursi, dan Radix Sort hasilnya berbeda dari big-o $O(N)$ karena jumlah pass-nya bergantung pada jumlah digit nilai maksimum (D), sehingga kompleksitas yang relevan adalah $O(N \times D)$ yang membuat rasio pertumbuhannya berbeda sedikit dari big-o.

MODUL 5: SEARCHING

SOAL 1

Searching

Tujuan Praktikum

1. Mahasiswa mampu mengimplementasikan algoritma searching (Linear Search dan Binary Search).
2. Mahasiswa mampu melakukan analisis performa menggunakan waktu eksekusi (s), jumlah perbandingan (comparisons), dan indeks hasil pencarian (index_found).
3. Mahasiswa memahami pengaruh distribusi data dan kondisi data terhadap efisiensi algoritma pencarian.

Ketentuan Tugas

- Implementasi dikerjakan secara modular menggunakan template yang disediakan.
- Dilarang keras menggunakan library pencarian bawaan seperti `std::find` atau `std::binary_search`. Semua algoritma harus ditulis manual.
- Dilarang menggunakan AI. Ini kalian harus analisis sendiri. Baca dulu dari sumber-sumber buku, materi, internet yang diberikan.
- Kalau segampang ini masih pakai AI, see you next meet aja.
- Data yang akan dicari berupa `vector<int>`.
- Waktu eksekusi diukur dalam satuan mikrosecond (μs).
- Jika elemen tidak ditemukan, `index_found` bernilai -1

Algoritma Searching

- Linear Search
- Binary Search

Instruksi Pengujian

Lakukan pengujian terhadap setiap algoritma menggunakan beberapa kondisi data berikut:

1. Data acak (random) — khusus Linear Search; Binary Search wajib diuji pada data terurut.
2. Data sudah terurut menaik (sorted ascending).

3. Data terurut menurun (sorted descending) — khusus Linear Search.

Untuk setiap kondisi data, lakukan pengujian dengan tiga skenario pencarian:

- Elemen ada di awal (best case position).
- Elemen ada di tengah.
- Elemen tidak ada dalam data (not found).

Hal yang Perlu Dianalisis

Untuk setiap algoritma, lakukan analisis terhadap aspek berikut:

1. Kompleksitas waktu:
 - Best Case
 - Average Case
 - Worst Case
2. Karakteristik algoritma:
 - Apakah algoritma memerlukan data terurut atau tidak.
 - Apakah algoritma bersifat iteratif atau dapat diimplementasikan secara rekursif.
 - Mekanisme utama algoritma dalam melakukan pencarian.
3. Analisis operasi:
 - Jumlah comparison yang dilakukan hingga elemen ditemukan atau seluruh data diperiksa.
 - Nilai `index_found` (indeks elemen yang ditemukan, atau -1 jika tidak ditemukan).
 - Pengaruh posisi elemen target terhadap jumlah comparison.
 - Pengaruh ukuran data (n) terhadap jumlah operasi pada masing-masing algoritma.
4. Analisis performa:
 - Perbandingan waktu eksekusi (s) antar algoritma pada kondisi data yang sama.
 - Algoritma mana yang lebih efisien untuk dataset kecil dan dataset besar.
 - Pengaruh kondisi data (terurut vs acak) terhadap performa masing-masing algoritma.
 - Hubungan antara teori kompleksitas ($O(n)$ vs $O(\log n)$) dan hasil eksperimen yang diperoleh.

A. Source Code

searching_algorithms.cpp

```
1  #include "searching_algorithms.h"
2  #include <chrono>
3
4  using Clock = std::chrono::high_resolution_clock;
5
6  void linearSearch(const std::vector<int> &data, int key,
7  Metrics &m) {
8      for (int i = 0; i < data.size(); i++) {
9          // cek satu persatu
10         m.comparisons++;
11         if (data[i] == key) {
12             m.found_index = i;
13             break;
14         }
15     }
16 }
17
18 void linear_search(const std::vector<int> &data, int
19 target, Metrics &m) {
20     auto start = Clock::now();
21
22     // Done: write the implementation here
23     linearSearch(data, target, m);
24
25     m.time_us =
26         std::chrono::duration<double,
27 std::micro>(Clock::now() - start).count();
28 }
29
30 void binarySearch(const std::vector<int> &data, int key,
31 Metrics &m) {
32     int low = 0;
33     int high = data.size() - 1;
34     while (low <= high) {
35         int mid = low + (high - low) / 2;
36
37         // cek apakah key ada di mid
38         m.comparisons++;
39         if (data[mid] == key) {
40             m.found_index = mid;
41             break;
42         } else if (data[mid] < key) {
43             low = mid + 1;
44         } else {
```

```

45     high = mid - 1;
46     }
47 }
48
49 // Jika tidak ada m.find_index nya default = -1
50 }
51
52 void binary_search(const std::vector<int> &data, int
53 target, Metrics &m) {
54     auto start = Clock::now();
55
56     // Done: write the implementation here
57     binarySearch(data, target, m);
58
59     m.time_us =
60         std::chrono::duration<double,
61 std::micro>(Clock::now() - start).count();
62 }

```

Table 25. Source Code searching_algorithms.cpp

B. Pembahasan

Linear Search

Kompleksitas waktu

1. **Best Case** : $O(1)$. Karena jika target berada di index 0 maka cuma perlu 1 comparison saja.
2. **Average Case** : $O(n)$. Karena rata – rata disini asumsi nya target berada sekitar tengah maka comparasion nya kurang lebih $(n/2)+1$.
3. **Worst Case** : $O(n)$. Karena worst disini artinya target tidak ada di vector data algoritma ini akan terus berjalan sampai $n-1$.

Untuk Big-O notation nya adalah $O(n)$. Karena algoritma ini melakukan for loop sebanyak n . Dengan hasil comparasi 1 di best case dan $target+1$ comparision di average case dan n comparison di worst.

Karakteristik algoritma

Karena algoritma ini melakukan for loop sebanyak n untuk dan membandingkan nya dengan target, maka algoritma ini tidak memerlukan data yang terurut. Algoritma ini bekerja dengan memalakukan iterasi terhadap setiap index dan membandingkan nya dengan target sehingga operasi utama yang digunakan adalah 1 loop sebanyak n dan 1 comparasi didalamnya.

Algoritma yang saya buat ini berbentuk iteratif. Namun algoritma ini bisa saja dibuat dengan rekursif namun ini membuatnya terasa lebih kompleks dari yang seharusnya. Ini contoh code rekursif nya.

Linear search Rekursif

```

1 void    linear_search_rekursif(const    std::vector<int>
2 &data, int target, int i,
3                                     int n, Metrics &m) {
4     // sudah lewat akhir data / nggak ketemu
5     if (i >= n) {
6         return; // found_index = default
7     }
8     // Ditaroh ditengah supaya jika sudah habis tidak kena
9     comparison
10    m.comparisons++;
11    // ketemu
12    if (data[i] == target) {
13        m.found_index = i;
14        return;
15    }
16    // rekursif
17    linear_search_rekursif(data, target, i + 1, n, m);
18 }

```

Table 26. Contoh linear search rekursif

Analisis operasi

Comparison : pada best case 1 dan pada worst case sebanyak n (semua data).

Found Index : found index pada best case adalah 1, dan -1 (tidak ketemu) pada worst case.

Pengaruh posisi target pada comparison : pengaruh nya adalah posisi+1, dimana found index 374, comparison 375.

Secara umum besarnya data tidak berpengaruh dalam menentukan banyaknya operasi karena pengaruh posisi target lebih dominan, pada best dan average case jika posisi target 20 dan data 1000, maka comparison nya tetap 21. Bahkan tidak berpengaruh dengan waktu nya, dimana pada index 0 dengan n=1000 time=0.130 , n=10000 time=0.040. Namun Besar data mempengaruhi worst case dimana target berada terakhir dengan n= 1000000 target=999999 comparison = 1000000 waktu = 5405.547. Jadi ya besar data hanya berpengaruh pada worst case.

Binary Search

Kompleksitas waktu

1. **Best Case** : $O(1)$. Jika posisi target ada di tengah vector maka cuma 1 komparasi dan kompleksitasnya jadi konstan.
2. **Average Case** : $O(\log n)$. Karena posisi target acak dan tidak ditengah vector maka range pencarian di vector akan terus dibagi 2 sampai target ketemu di tengah range itu. Dimana hasil nya diharapkan kurang dari $\log n$.
3. **Worst Case** : $O(\log n)$. Karena data tidak ada di vector pada algoritma ini hanya bisa terjadi jika target melebihi n . maka worst case nya sudah pasti $\log n$ pas

Karena algoritma ini merupakan divide and conquer dimana vector data akan dibagi 2 maka secara umum Big-O notation nya jika tidak ada variable lain yang mempengaruhinya adalah $O(\log n)$.

Karakteristik algoritma

Karena algoritma ini membandingkan nilai index mid pada range vector terhadap target untuk menentukan besaran range nya dia memerlukan data yang berurut, sebagai contoh jika target lebih kecil dari mid maka high (belakang) diturunkan ke posisi mid, dan mid akan dikalkulasi ulang. Dari sini saja sudah terlihat bahwa algoritma ini perlu data yang urut. Karena itu operasi utama dari algoritma ini adalah 1 loop while (jika rekursi tidak ada ini) dan 1 komparasi. Dimana loop while sebagai iterasi utama dan komparasi untuk cek target terhadap relatif posisi nya dari mid dan juga mengubah range pencarian.

Algoritma yang saya buat di kode ini bersifat iteratif, tetapi ya algoritma ini ada versi rekursif nya juga. Namun perbedaan nya disini adalah rekursif memiliki kompleksitas ruang $O(\log n)$ karena dia akan rekursif sebanyak $\log n$ (data vector dibagi 2 terus) dan ini menimbulkan keperluan memori di callstack. Dan ini contoh fungsi nya :

Binary search Rekursif

```

1 void    binary_search_rekursif(const    std::vector<int>
2 &data, int target, int low,
3                                     int high, Metrics &m) {
4     // ruang pencarian habis / tidak ketemu
5     if (low > high) {
6         return; // found_index default
7     }
8     int mid = low + (high - low) / 2;
9     m.comparisons++;
10    if (data[mid] == target) {
11        m.found_index = mid;
12        return;
13    } else if (data[mid] < target) {
14        // Cari ke kanan
15        binary_search_rekursif(data, target, mid + 1, high,
```

```

16 m);
17 } else {
18     // Cari ke kiri
19     binary_search_rekursif(data, target, low, mid - 1,
20 m);
21 }
22 }

```

Table 27. Contoh Binary search rekursif

Analisis operasi

Comparison : pada best case 1 dan pada worst case sebanyak $\log n$ (semua data).

Found Index : found index pada best case adalah 1, dan -1 (tidak ketemu) pada worst case.

Pengaruh posisi target pada comparison : posisi data cukup berpengaruh terhadap banyaknya operasi karena jika target berada di tengah vector atau pada $n/2!$ (n /anggota dari faktorial 2) yang factor nya tidak terlalu tinggi maka level rekursi / iterasi nya tidak terlalu banyak.

Besarnya data tidak berpengaruh dalam menentukan banyaknya operasi karena pengaruh posisi target lebih dominan, namun sama dengan linear banyak data disini mempengaruhi worst case tetapi untuk binary karena notasinya $\log n$ data besar akan teratasi. Contoh $n = 1000000$ comparasinya cuma = 20.

Analisis performa

	Linear Search	Binary Search
Average N=1000 Comp	375	3
Average N=1000 Time	2.044 us	0.080 us
Worst N=1000 Comp	1000	10
Worst N=1000 Time	5.350 us	0.130 us
Average N=1000000 Comp	374541	15
Average N=1000000 Time	2042.802 us	0.290 us
Worst N=1000000 Comp	1000000	20
Worst N=1000000 Time	5405.547 us	0.251 us

Table 28. Table hasil experiment Modul 5

Waktu Linear Search jauh lebih lambat dari Binary Search untuk besar seperti $N=1.000.000$, Linear bisa ratusan dan ribuan mikrodetik sedangkan Binary cuma sekitar dibawah satu mikrodetik. Jauh banget beda waktu untuk kedua algoritma ini untuk dataset besar. Tapi untuk N kecil (misal 1000), beda nya tidak terlalu jauh

Untuk dataset kecil linear masih lumayan, namun yang menjadi keunggulan nya adalah dia tidak perlu data yang terurut. Untuk dataset besar binary search jauh lebih baik. Perbedaan antara $O(N)$ dan $O(\log N)$ akan sangat terasa saat n nya jutaan.

Pengaruh case pada linear search terasa di worst case (target tidak ada) membuat comparisons naik drastis jadi n . Sedangkan pada binary search worst case comparisons cuma naik jadi $\log N$, dimana disini sangat kecil.

Berdasarkan table hasil experiment diatas bisa kita simpulkan bahwa untuk linear search pertumbuhan waktu nya mendekati $O(n)$ dan comparison nya adalah di $O(n)$. Sedangkan waktu binary search hampir konstan meskipun N bertambah besar. Saat N dilipatkan 1000 kali dari 1.000 ke 1.000.000, waktu hanya naik sekitar 1.9x sedangkan comparison nya naik 2x. Ini sangat sesuai dengan teori $O(\log n)$ di mana pertumbuhan waktu sangat lambat seiring bertambahnya N .

MODUL 6: TREE AND GRAPH

SOAL 1

Konversi Adjacency List ke Adjacency Matrix

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U,V,W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U,V,W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

F. N adalah jumlah vertex.

G. V_i adalah label vertex ke- i .

H. M adalah jumlah edge.

I. U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .

J. W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

V1 V2 ... VN

V1 a11 a12 ... a1N

V2 a21 a22 ... a2N

...

VN aN1 aN2 ... aNN

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

A. $2 \leq N \leq 10$

B. $0 \leq M \leq N \cdot (N-1)$

C. $1 \leq W_i \leq 1000$

D. Label vertex berupa karakter alfabet kapital A sampai Z.

E. Semua label vertex dijamin unik.

F. Graph tidak memiliki self-loop.

G. Graph tidak memiliki edge duplikat dengan arah yang sama.

H. Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5	Adjacency Matrix:
A B C D E	A B C D E
6	A 0 4 2 0 0
A B 4	B 0 0 0 7 0
A C 2	C 0 1 0 0 6
B D 7	D 0 0 3 0 0
C B 1	E 0 0 0 0 0
C E 6	

Table 29. Contoh input dan output soal 1 Modul 6

A. Source Code

soal1.cpp

```

1  #include <iostream>
2  #include <map>
3  #include <vector>
4  using namespace std;
5
6  int main() {
7
8      // Vertex / node
9      int N;
10     cin >> N;
11
12     vector<char> label(N);
13     for (int i = 0; i < N; i++) {
14         cin >> label[i];
15     }
16
17     // Buat mapping char
18     map<char, int> adj;
19     for (int i = 0; i < N; i++) {
20         adj[label[i]] = i;
21     }
22
23     // Inisialisasi matrix N x N dengan 0
24     vector<vector<int>> matrix(N, vector<int>(N, 0));
25
26     // Edge / Busur / panah
27     int M;
28     cin >> M;
29
30     // Isi matrix
31     for (int i = 0; i < M; i++) {
32         char u, v;
33         int w;
34         cin >> u >> v >> w;
35         matrix[adj[u]][adj[v]] = w;
36     }
37
38     // Cetak text header
39     cout << "Adjacency Matrix:" << endl;
40

```

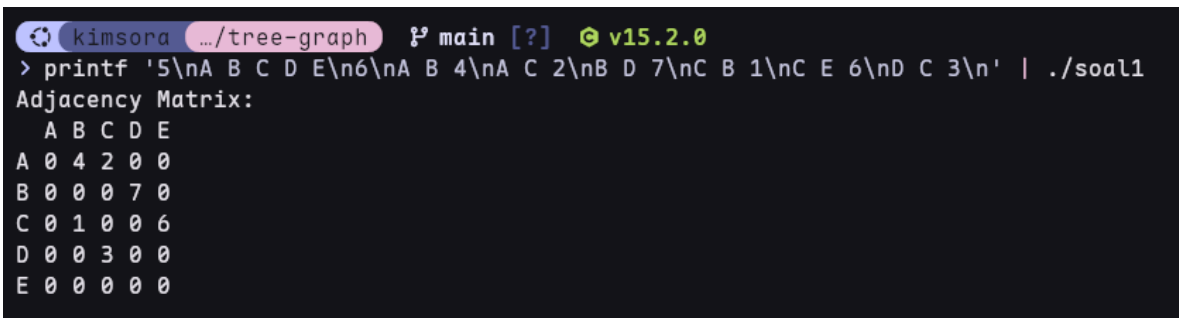
```

41 // Cetak header matrix
42 cout << " ";
43 for (int i = 0; i < N; i++) {
44     if (i > 0)
45         cout << " ";
46     cout << label[i];
47 }
48 cout << endl;
49
50 // Cetak setiap baris matrix
51 for (int i = 0; i < N; i++) {
52     cout << label[i];
53     for (int j = 0; j < N; j++) {
54         cout << " " << matrix[i][j];
55     }
56     cout << endl;
57 }
58
59 return 0;
60 }

```

Table 30. Source Code soal1.cpp Modul 6

B. Output Program



```

kimsora .../tree-graph main [?] v15.2.0
> printf '5\nA B C D E\n6\nA B 4\nA C 2\nB D 7\nC B 1\nC E 6\nD C 3\n' | ./soal1
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0

```

Gambar 7. Screenshot Hasil Jawaban Soal 1 Modul 6

C. Pembahasan

Kompleksitas waktu

Karena disini ada dua variable yang menjadi patokan seberapa banyak perulangan akan terjadi yaitu N (banyak node) dan M (banyak edge / panah) maka kompleksitas waktu nya adalah $O(n \cdot m)$.

Kompleksitas ruang

Untuk menampung node diperlukan vector dengan size N , dan ada juga vector $N \times N$ (nested vector) yang digunakan sebagai penyimpan matrix, disini perbedaannya tidak ada ruang yang bergantung pada M . Ini menjadikan kompleksitas ruangnya menjadi $O(n^2)$.

Karakteristik problem

Problem ini meminta untuk mengubah adjacency list menjadi matriks adjacency. Kita perlu menyimpan input banyaknya node dan juga labelnya karena itu kita menyimpan banyaknya node dan juga label untuk setiap node dengan vector char. Kemudian kita lakukan mapping label agar memiliki urutan dengan map<char, int>. Untuk memasukan berapa banyaknya edge yang ingin dibuat kita simpan pada variable m , setelah ini kita perlu lakukan loop untuk memasukan list dan sekaligus mengubahnya menjadi nilai di matriks, dengan cara $matrix[adj[u]][adj[v]] = w$.

SOAL

SOAL 2

Konversi Adjacency Matrix ke Adjacency List

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

K. N adalah jumlah vertex.

L. V_i adalah label vertex ke- i .

M. A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .

N. Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .

O. Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency List:

$V_1 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

$V_2 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Batasan

- A. $2 \leq N \leq 10$
- B. $0 \leq A_{ij} \leq 1000$
- C. Label vertex berupa karakter alfabet kapital A sampai Z.
- D. Semua label vertex dijamin unik.
- E. Nilai diagonal utama matrix dijamin 0.
- F. Nilai 0 berarti tidak ada edge.
- G. Nilai positif berarti bobot edge.
- H. Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E -> -

Table 31. Contoh input dan output soal 2 Modul 6

A. Source Code

soal2.cpp

1	#include <iostream>
2	#include <vector>
3	using namespace std;
4	
5	int main() {
6	// Banyak Label

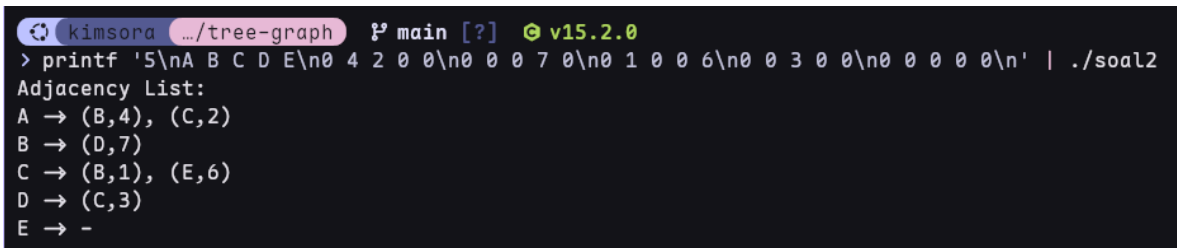
```

7   int N;
8   cin >> N;
9
10  // Baca Label Karakter
11  vector<char> label(N);
12  for (int i = 0; i < N; i++) {
13      cin >> label[i];
14  }
15
16  // Baca Matrix adjacency
17  vector<vector<int>> matrix(N, vector<int>(N));
18  for (int i = 0; i < N; i++) {
19      for (int j = 0; j < N; j++) {
20          cin >> matrix[i][j];
21      }
22  }
23
24  // Cetak header dan matrix adjacency list
25  cout << "Adjacency List:" << endl;
26  for (int i = 0; i < N; i++) {
27      cout << label[i] << " -> ";
28      bool hasEdge = false;
29      for (int j = 0; j < N; j++) {
30          if (matrix[i][j] > 0) {
31              if (hasEdge) {
32                  cout << ", ";
33              }
34              cout << "(" << label[j] << ", " << matrix[i][j] <<
35  ")\n";
36              hasEdge = true;
37          }
38      }
39      if (!hasEdge) {
40          cout << "-\n";
41      }
42      cout << endl;
43  }
44
45  return 0;
46 }

```

Table 32. Source Code soal2.cpp Modul 6

B. Output Program



```
kimsora .../tree-graph main [?] v15.2.0
> printf '5\nA B C D E\n0 4 2 0 0\n0 0 0 7 0\n0 1 0 0 6\n0 0 3 0 0\n0 0 0 0 0\n' | ./soal2
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
```

Gambar 8. Screenshot Hasil Jawaban Soal 2 Modul 6

C. Pembahasan

Kompleksitas waktu

Untuk membaca matriks yang diberikan pada input kita perlu melakukan nested loop yang bergantung dengan banyak nya N (node), dan untuk mencetak adjacency list kita juga perlu melakukan nested loop yang bergantung terhadap N juga, jadi ini membuat kompleksitas waktu nya adalah Big-O notation $O(n^2)$.

Kompleksitas ruang

Sama seperti soal 1 kita masih memerlukan nested vector yang kedua nya sebanyak N (matriks $N \times N$) jadi kompleksitas ruang nya sama yaitu Big-O notation $O(n^2)$.

Karakteristik problem

Sama seperti sebelum nya problem ini memerlukan kita untuk membaca label node, yang membedakan nya disini kita perlu menerima input matriks, untuk menerima itu kita perlu nested vector sebagai wadah matriks, dan nested loop untuk mengisi value setiap cell pada matriks dengan cin `>> matrix[i][j]`. Inti dari problem ini adalah mengubah matriks menjadi adjacency list disini lah kita akan transform dari matriks ke list dengan cara kita loop sebanyak N untuk setiap label, baru didalam nya ada loop lagi untuk menentukan apakah node ini ada edge nya diperlukan komparasi `matrix[i][j] > 0` dan cetak edge yang ditemukan.

SOAL 3

BFS: Jarak Terpendek Keluar Labirin

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

S_R S_C

F_R F_C

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.
- Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak adjacency matrix dengan format berikut.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Batasan

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0	16

0 0	
7 9	

Table 33. Contoh input dan output soal 3 Modul 6

A. Source Code

soal3.cpp

```

1  #include <iostream>
2  #include <queue>
3  #include <vector>
4  using namespace std;
5
6  int main() {
7      int R, C;
8      cin >> R >> C;
9
10     // grid R x C
11     vector<vector<int>> grid(R, vector<int>(C));
12     for (int i = 0; i < R; i++) {
13         for (int j = 0; j < C; j++) {
14             cin >> grid[i][j];
15         }
16     }
17
18     // posisi awal dan tujuan
19     int sr, sc, fr, fc;
20     cin >> sr >> sc;
21     cin >> fr >> fc;
22
23     // Inisialisasi visited dan dist
24     vector<vector<bool>> visited(R, vector<bool>(C,
25 false));
26     vector<vector<int>> dist(R, vector<int>(C, -1));
27
28     // Definisikan 4 arah pergerakan
29     int dr[] = {-1, 1, 0, 0};
30     int dc[] = {0, 0, -1, 1};
31
32     // Inisialisasi queue dan push start
33     queue<pair<int, int>> q;
34     q.push({sr, sc});
35     visited[sr][sc] = true;
36     dist[sr][sc] = 0;
37
38     // BFS Loop

```

```

39 while (!q.empty()) {
40     auto [r, c] = q.front();
41     q.pop();
42
43     // Cek apakah sudah sampai tujuan
44     if (r == fr && c == fc) {
45         cout << dist[r][c] << endl;
46         return 0;
47     }
48
49     // Cek 4 arah
50     for (int i = 0; i < 4; i++) {
51         int nr = r + dr[i];
52         int nc = c + dc[i];
53
54         // Validasi batas grid + bukan dinding + belum
55         visited
56         if (nr >= 0 && nr < R && nc >= 0 && nc < C &&
57         grid[nr][nc] == 0 &&
58         !visited[nr][nc]) {
59             visited[nr][nc] = true;
60             dist[nr][nc] = dist[r][c] + 1;
61             q.push({nr, nc});
62         }
63     }
64 }
65 cout << -1 << endl;
66 return 0;
67 }

```

Table 34. Source Code soal3.cpp Modul 6

B. Output Program



```

kimsora .../tree-graph P main [?] v15.2.0
> printf '8 10\n0 0 0 0 1 0 0 0 0 0\n1 1 1 0 1 0 1 1 1 0\n0 0 1 0 0 0 0 0 1 0\n0 1 1 1 1 1
1 0 1 0\n0 0 0 0 0 0 1 0 0 0\n1 1 1 1 1 0 1 1 1 0\n0 0 0 0 1 0 0 0 1 0\n0 1 1 0 0 0 1 0 0 0
\n0 0\n7 9\n' | ./soal3
16

```

Gambar 9. Screenshot Hasil Jawaban Soal 3 Modul 6

C. Pembahasan

Kompleksitas waktu

Karena kita membaca grid jadi kompleksitas waktu nya adalah $O(r \cdot c)$ sedangkan untuk bfs nya sendiri juga adalah $O(r \cdot c)$ pada worst case karena dia akan melakukan loop semua node.

Kompleksitas ruang

Kompleksitas ruang nya disini adalah $O(r \cdot c)$ karena ada 3 buah vector $R \times C$ yang selalu penuh dan 1 queue yang worst case bisa mencapai $O(r \cdot c)$ juga. Jadi Big-O notation nya $O(r \cdot c)$

Karakteristik problem

Problem ini adalah sebuah labirin yang direpresentasikan oleh grid berukuran $R \times C$. dengan isi 0 jika jalan bisa dilewati dan 1 jika itu dinding yang tidak bisa dilewati, kita diminta untuk mencari jumlah langkah minimal untuk mencapai tujuan, dengan syarat kita tidak bisa bergerak diagonal. Artinya kita diminta menggunakan Breadth-First Search. Dan input nya adalah R, C, Matriks, Koordinat awal, Koordinat tujuan.

Untuk R dan C standart saja input simpan di variable, untuk matriks juga standart input disimpan dengan cara nested loop sesuai index nya. Hal yang penting disini adalah kita perlu membuat grid berukuran sama yang menyimpan nilai true false sebagai penunjuk apakah langkah sudah dilalui. Kita juga membuat satu grid berukuran sama lagi untuk mencatat jarak yang tengah dilalui saat ini. Sebelum kita mulai loop utama bfs kita perlu init queue $\text{pair}\langle \text{int}, \text{int} \rangle$ dan push sr, sc kedalam nya, set $\text{visited}[sr][sc] = \text{true}$ dan $\text{dist}[sr][sc] = 0$. Kita juga perlu mendefinisikan 4 arah pergerakan dengan cara membuat 2 array yaitu dr (row) dc (kolom) dimana $dr[] = \{-1, 1, 0, 0\}$; yang menandakan jika bergerak ke atas row nya -1, jika kebawah row nya +1, dan $dc[] = \{0, 0, -1, 1\}$; yang menandakan jika bergerak kiri kolom nya -1 jika kanan kolom nya +1 (index / kordinat).

Pada loop bfs dengan kondisi $\text{while} (!q.empty())$ ini kita perlu mengambil nilai koordinat dari pair yang ada di front queue dengan cara deconstructor menjadi r dan c , kemudian kita $\text{pop} / \text{dequeue}$, karena kita sudah mengambil nilainya dan akan melangkah dari titik ini jadi titik ini tidak perlu disimpan di queue. Kemudian kita lakukan cek apakah koordinat saat ini sudah sama dengan koordinat tujuan jika sudah cetak nilai dari vector dist dengan koordinat saat ini. Nah disini kita akan melakukan langkah atau cek 4 arah dengan cara loop 4x disini kita simpan kordinat baru nya dengan cara $nr = r + dr[i]$ dan $nc = c + dc[i]$. dc dan dr berfungsi sebagai pengubah kordinat (langkah) yang sudah kita tentukan sebelumnya. Setelah itu dilakukan validasi batas grid + bukan dinding + belum visited, dan jika kordinat saat ini valid baru kita tandai kordinat ini true di visited grid dan juga simpan distace nya di grid dist dengan cara $\text{dist}[nr][nc] = \text{dist}[r][c] + 1$; baru kita push kordinat ini sebagai pair kedalam queue.

Dan ini akan diulang terus sampai queue empty atau sudah sampai tujuan. Jadi loop akan mengambil nilai pair pada queue depan dan dequeue nya, lalu lakukan apakah nilai dari pair

sudah sama dengan tujuan? Jika ya print jarak, jika tidak proses 4 langkah, langkah yang valid akan masuk ke queue.

SOAL 4

DFS: Menghitung Semua Jalur Keluar Labirin

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

$R\ C$

$G_{11}\ G_{12}\ \dots\ G_{1C}$

$G_{21}\ G_{22}\ \dots\ G_{2C}$

...

$G_{R1}\ G_{R2}\ \dots\ G_{RC}$

$S_R\ S_C$

$F_R\ F_C$

Keterangan:

- A. R adalah jumlah baris grid.
- B. C adalah jumlah kolom grid.
- C. G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- D. (S_R, S_C) adalah posisi awal.
- E. (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak adjacency matrix dengan format berikut.

T

Keterangan:

F. T adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.

G. Jika posisi tujuan tidak dapat dicapai, cetak 0.

Batasan

A. $1 \leq R \leq 7$

B. $1 \leq C \leq 7$

C. G_{ij} hanya bernilai 0 atau 1.

D. $0 \leq S_R < R$

E. $0 \leq S_C < C$

F. $0 \leq F_R < R$

G. $0 \leq F_C < C$

H. Sel start dan finish dijamin bernilai 0.

I. Pergerakan hanya boleh dilakukan ke sel bernilai 0.

J. Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.

K. Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.

L. Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Table 35. Contoh input dan output soal 4 Modul 6

A. Source Code

soal4.cpp

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void dfs(int r, int c, int fr, int fc, int R, int C,
6           const vector<vector<int>> &grid,
7           vector<vector<bool>> &visited,
8           int &pathCount) {
9
10     if (r == fr && c == fc) {
11         pathCount++;
12         return;
13     }
14     visited[r][c] = true;
15
16     // Petunjuk 4 arah
17     int dr[] = {-1, 1, 0, 0};
18     int dc[] = {0, 0, -1, 1};
19
20     // Cek 4 arah
21     for (int i = 0; i < 4; i++) {
22         int nr = r + dr[i];
23         int nc = c + dc[i];
24         // Validasi
25         if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr]
26 [nc] == 0 &&
27         !visited[nr][nc]) {
28             dfs(nr, nc, fr, fc, R, C, grid, visited,
29 pathCount);
30         }
31     }
32     // Backtrack
33     visited[r][c] = false;
34 }
35
36 int main() {
37     int R, C;
38     cin >> R >> C;
39
40     vector<vector<int>> grid(R, vector<int>(C));
41     for (int i = 0; i < R; i++) {
42         for (int j = 0; j < C; j++) {
43             cin >> grid[i][j];
44         }
```

```

45     }
46
47     int sr, sc, fr, fc;
48     cin >> sr >> sc;
49     cin >> fr >> fc;
50
51     vector<vector<bool>> visited(R, vector<bool>(C,
52 false));
53     int pathCount = 0;
54     dfs(sr, sc, fr, fc, R, C, grid, visited, pathCount);
55     cout << pathCount << endl;
56
57     return 0;
58 }

```

Table 36. Source Code soal4.cpp Modul 6

B. Output Program



```

kimsora ~/tree-graph main [?] v15.2.0
> printf '5 6\n0 0 1 0 0\n0 1 0 0 1\n0 1 0 1 0\n0 0 1 1 0\n1 1 0 0 0\n0 0\n4 5\n'
| ./soal4
4

```

Gambar 10. Screenshot Hasil Jawaban Soal 4 Modul 6

C. Pembahasan

Kompleksitas waktu

Karena algoritma yang digunakan adalah dfs + backtracking untuk mencari semua jalur yang ada, berbeda dengan dfs normal yang kompleksitas nya adalah $O(r \cdot c)$ / $O(n^2)$. Karena backtracking ini akan membuat nya kembali satu langkah dan mencoba langkah lain, dan ini membuat kompleksitas nya menjadi eksponensial, untuk menentukan kompleksitas waktu nya kita perlu mengetahui kondisi dari case kita yaitu langkah 4 arah, dan tidak bisa langkah mundur yang membuat kita mempunyai 3 case setiap langkah. Dan kita perlu melakukan ini untuk semua node pada grid yang menjadikan kompleksitas waktunya adalah $O(3^{R \times C})$. Karena pada worst case nya akan ada 3 case untuk setiap node.

Kompleksitas ruang

Yah masih sama seperti soal sebelumnya memiliki 2 vector grid $R \times c$ yang membuat nya $O(r \cdot c)$, yang membedakan nya disini adalah adanya recursif pemanggilan yang membuat call stack yang bergantung kepada R dan C. karena kedua nya memiliki kompleksitas $O(r \cdot c)$

Karakteristik problem

Sama seperti soal 3 kita masih di labirin. Namun disini kita diminta untuk menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search. Dengan aturan tidak boleh mengunjungi sel yang sama lebih dari 1 kali dan tidak boleh bergerak diagonal cuman 4 arah. Input yang diterima juga sama dengan soal sebelumnya.

Untuk menyelesaikan problem ini, kita memerlukan sebuah array visited berukuran sama dengan grid yang bersifat sementara. Berbeda dengan soal sebelumnya di mana visited bersifat permanen setelah sebuah sel diproses, di sini visited harus bisa dikembalikan setelah selesai menelusuri satu cabang jalur. Prosesnya dimulai dengan memanggil fungsi rekursif dfs dari koordinat awal. Di dalam fungsi ini, pertama kita periksa apakah posisi saat ini sudah sama dengan koordinat tujuan. Jika ya, kita tingkatkan nilai counter jalur sebanyak satu lalu return.

Sebelum melanjutkan ke empat arah pergerakan, koordinat saat ini harus ditandai sebagai visited agar tidak terjadi pengulangan sel dalam satu jalur yang sedang ditelusuri. Kemudian untuk masing-masing dari empat arah yang mungkin, kita hitung koordinat baris dan kolom baru menggunakan pair array dr dan dc yang menentukan . Setiap koordinat baru yang lolos validasi, yaitu berada dalam batas grid, bukan dinding, dan belum dikunjungi pada jalur ini, akan menjadi titik awal pemanggilan rekursif berikutnya.

Ketika pemanggilan rekursif untuk suatu arah selesai dan return, yang terjadi adalah kita telah menyelesaikan eksplorasi seluruh cabang yang dimulai dari sel tersebut. Pada titik inilah mekanisme backtracking berperan penting: kita harus mengubah status visited untuk koordinat saat ini kembali menjadi false. Tindakan ini memberikan kesempatan pada jalur-jalur lain yang mungkin melintasi sel yang sama namun melalui rute berbeda untuk tetap bisa dieksplorasi. Tanpa langkah ini, program hanya akan menemukan satu jalur pertama dan mengabaikan semua kemungkinan alternatif.

Proses rekursi dan backtracking ini terus berlanjut hingga seluruh kemungkinan jalur sederhana telah diperiksa. Counter jalur yang telah ditambah setiap kali mencapai tujuan kemudian dicetak sebagai output akhir. Jika tidak ada satupun jalur yang valid, counter tetap bernilai nol dan itulah yang akan ditampilkan.

SOAL 5

Tree Traversal

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red-Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- A. PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- B. INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- C. POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- D. ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

...

T_Q

Keterangan:

- H. N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- I. A_i adalah bilangan ke- i .
- J. Q adalah banyaknya query traversal.
- K. T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Batasan

A. $0 \leq N \leq 1000$

B. $-10000000000 \leq A_i \leq 10000000000$

C. $1 \leq Q \leq 20$

D. T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

Table 37. Contoh input dan output soal 5 Modul 6

A. Source Code

soal5.cpp

1	#include "RedBlackTree.h"
2	#include <iostream>
3	#include <string>

```

4 using namespace std;
5
6 void preorderHelper(const RedBlackTree::Node *node,
7                    const RedBlackTree::Node *nil) {
8     if (node == nil || node->isNil)
9         return;
10    cout << node->key << " ";
11    preorderHelper(node->left, nil);
12    preorderHelper(node->right, nil);
13 }
14
15 void inorderHelper(const RedBlackTree::Node *node,
16                  const RedBlackTree::Node *nil) {
17     if (node == nil || node->isNil)
18         return;
19     inorderHelper(node->left, nil);
20     cout << node->key << " ";
21     inorderHelper(node->right, nil);
22 }
23
24 void postorderHelper(const RedBlackTree::Node *node,
25                    const RedBlackTree::Node *nil) {
26     if (node == nil || node->isNil)
27         return;
28     postorderHelper(node->left, nil);
29     postorderHelper(node->right, nil);
30     cout << node->key << " ";
31 }
32
33 void preorder(const RedBlackTree &tree) {
34     preorderHelper(tree.root(), tree.nil());
35 }
36
37 void inorder(const RedBlackTree &tree) {
38     inorderHelper(tree.root(), tree.nil());
39 }
40
41 void postorder(const RedBlackTree &tree) {
42     postorderHelper(tree.root(), tree.nil());
43 }
44
45
46 int main() {
47     // Banyak nya Node
48     int N;
49     cin >> N;
50
51     RedBlackTree tree;

```

```

52
53 // Insert node
54 for (int i = 0; i < N; i++) {
55     int val;
56     cin >> val;
57     if (!tree.contains(val)) {
58         tree.insert(val);
59     }
60 }
61
62 if (tree.empty()) {
63     cout << "Tree kosong. Tidak ada yang bisa
64 ditampilkan." << endl;
65     return 0;
66 }
67
68 // Banyaknya Langkah Travers
69 int Q;
70 cin >> Q;
71
72 for (int i = 0; i < Q; i++) {
73     string query;
74     cin >> query;
75     if (query == "PREORDER") {
76         cout << "[Preorder] : ";
77         preorder(tree);
78         cout << endl;
79     } else if (query == "INORDER") {
80         cout << "[Inorder] : ";
81         inorder(tree);
82         cout << endl;
83     } else if (query == "POSTORDER") {
84         cout << "[Postorder] : ";
85         postorder(tree);
86         cout << endl;
87     } else if (query == "ALL") {
88         cout << "[Preorder] : ";
89         preorder(tree);
90         cout << endl;
91         cout << "[Inorder] : ";
92         inorder(tree);
93         cout << endl;
94         cout << "[Postorder] : ";
95         postorder(tree);
96         cout << endl;
97     }
98 }
99

```

100	return 0;
101	}

Table 38. Source Code soal5.cpp Modul 6

B. Output Program

```

kimsora .../tree-graph  main [?]  v15.2.0
> printf '7\n50 30 70 20 40 60 80\n4\nPREORDER\nINORDER\nPOSTORDER\nALL\n' | ./soal5
[Preorder] : 50 30 20 40 70 60 80
[Inorder]   : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder]   : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50

```

Gambar 11. Screenshot Hasil Jawaban Soal 5 Modul 6

C. Pembahasan

Kompleksitas waktu

Metode insert dan contains yang diberikan pada rbt memiliki kompleksitas $O(\log n)$ dan traversal yang akan selalu mengunjungi setiap node maka kompleksitas nya $O(n)$ namun traversal disini dijalankan di atas loop Q yang menjadikan Kompleksitas gabungannya adalah $O(q n)$.

Kompleksitas ruang

Karena rekursi traversal mendalam maksimal tinggi tree membentuk call stack = $O(\log N)$ karena RBT nya seimbang. Dan node pada tree sendiri sebanyak $O(n)$ maka $O(\log N) + O(n)$ tetap membuatnya menjadi $O(n)$

Karakteristik problem

Problem ini adalah manipulasi struktur data binary tree yang self-balancing, di mana kita diminta untuk membangun Red-Black Tree dari sekumpulan bilangan masukan kemudian menjawab beberapa query berupa permintaan hasil traversal tertentu. Berbeda dengan soal-soal sebelumnya yang lebih kepada representasi graph dan eksplorasi jalur, di sini inti permasalahannya adalah bagaimana memanfaatkan sifat BST yang terjaga balanced-nya untuk menghasilkan urutan node sesuai jenis traversal yang diminta.

Kita bisa memanfaatkan method publik yang tersedia, yaitu root() dan nil() untuk mengakses struktur internal tree yang disediakan. Dua method ini memberikan kita pointer ke node root

dan sentinel nil, yang kemudian digunakan sebagai titik masuk untuk menulis fungsi traversal rekursif.

Ide utama penyelesaian dimulai dengan membaca N bilangan dan memasukkannya satu per satu ke dalam tree. Namun ada aturan khusus: duplikat harus diabaikan. Karena method `insert()` pada class yang diberikan tidak secara eksplisit menangani duplikat, maka sebelum melakukan insert kita perlu memastikan bahwa nilai yang akan dimasukkan belum ada di dalam tree dengan memanggil `contains()`. Jika `contains()` mengembalikan false, barulah `insert()` dieksekusi. Mekanisme ini menjaga agar setiap key dalam tree bersifat unik, yang merupakan prasyarat agar hasil traversal InOrder selalu menghasilkan urutan terurut dari kecil ke besar. Tanpa penanganan duplikat ini, struktur BST bisa menjadi ambigu dan hasil traversal mungkin tidak konsisten.

Setelah tree terbentuk, program membaca Q query. Setiap query berupa string yang menentukan jenis traversal yang diminta. Karena traversal yang digunakan adalah rekursif, kita memerlukan tiga pasang fungsi helper yang bekerja dengan pola visit yang berbeda. Fungsi preorder mencetak node saat ini sebelum rekursi ke subtree kiri dan kanan, fungsi inorder menunda pencetakan hingga seluruh subtree kiri selesai diproses, dan fungsi postorder menunda hingga kedua subtree selesai. Semua fungsi ini menggunakan nil sebagai kondisi terminasi, bukan `nullptr`, karena implementasi `RedBlackTree` yang diberikan menggunakan sentinel node untuk menandakan akhir cabang. Penggunaan pengecekan `node->isNil` atau `node == nil` menjadi kritis agar traversal berhenti pada leaf tanpa menyebabkan infinite recursion.

Output diformat sedemikian rupa sesuai jenis query. Ketika query bernilai ALL, ketiga traversal dicetak secara berurutan. Seluruh proses ini mengandalkan fakta bahwa Red-Black Tree tetap mempertahankan sifat BST meskipun dilakukan rotasi dan recoloring selama insert, sehingga urutan InOrder selalu sorted ascending dan urutan PreOrder serta PostOrder selalu konsisten dengan struktur tree yang terbentuk pada saat itu.

TAUTAN GIT

Berikut adalah tautan untuk semua source code yang telah dibuat.

<https://github.com/biahli/Algoritma-dan-Struktur-Data.git>